

Eötvös Loránd University

Faculty of Informatics

Dept. of Artificial Intelligence

Design and Implementation of AkVen Integrated E-Commerce and Inventory Management System

Supervisor:
Walid Guettala
PhD candidate, MSc

Author:
Nurmuhammed Kanybekov
Computer Science BSc

Budapest, 2026

EÖTVÖS LORÁND UNIVERSITY
FACULTY OF INFORMATICS

Thesis Registration Form

Attention! The deadline for amendment is 1 February in case of final exam in June and 31 August in case of final exam in January.

Student's Data:
Student's Name: Kanybekov Nurmuhammed
Student's Neptun code: NRTKMF

Educational Information:
Training programme: Computer Science BSc
I have an internal supervisor

Internal Supervisor's Name: *Walid Guettala*
Supervisor's Home Institution: ***Department of Artificial Intelligence***
Address of Supervisor's Home Institution: ***1117, Budapest, Pázmány Péter sétány 1/C.***

Supervisor's Position and Degree: *PhD candidate, MSc*

Thesis Title: Design and Implementation of AkVen Integrated E-Commerce and Inventory Management System

Topic of the Thesis:
(Upon consulting with your supervisor, give a 150-300-word-long synopsis os your planned thesis. Please do not use the "&" character in the text, otherwise the request will not generate correctly.)

Motivation
AkVen Socks is our family business specializing in the import and distribution of high quality socks from the renowned NabiYa manufacturer. Currently, stock tracking, order processing, and sales analysis are handled manually or with basic tools. This leads to frequent inventory discrepancies, delayed order fulfillment, and insufficient visibility into business performance.
I plan to design and implement a custom full stack web application for my family business, AkVen Socks, the official distributor of premium NabiYa socks. The system will consist of a modern customer facing ecommerce website and a secure administrative dashboard for managing products, inventory, orders, and sales. The application will be developed from scratch following a strict three layer architecture consisting of frontend, backend with business logic, and database layers.

Project Idea
The thesis will involve the development of a complete custom ecommerce and inventory management system. The application will include two main components:
A modern online store with a clean, visually appealing design and intuitive user interface where customers can easily browse, filter and buy socks
A secure administrative dashboard for managing the entire business operations
The system will be built entirely from scratch without using any content management platforms such as WordPress or Shopify. The primary objective is to replace the existing manual processes with an efficient, scalable, and maintainable software solution.

Main Features
Customer facing website:
Product catalog with advanced filtering by color, size, price, collection and other attributes
Detailed product pages with multiple images and variant selection
User registration, login, and personal order history
Shopping cart and checkout functionality

Admin dashboard:
Full CRUD operations for products including images and variants
Real time inventory management with automatic low stock alerts
Order management including viewing and status updates
Simple sales analytics with visual charts such as best selling items and revenue trends

Additional technical features:
Fully responsive design for both desktop and mobile devices
Secure authentication and role based access control

Technical Architecture:
The system will follow the classic three layer architecture:
Frontend layer using a modern frontend framework
Business logic layer containing core application rules for orders, inventory and alerts

Database layer with a well designed relational database and normalized schema

Budapest, 2026. 10. 10.

Abstract

This thesis presents the design and implementation of AK&VEN, an integrated e-commerce and inventory management system for a family business that has sold Korean-made socks from container 70-E at Dordoi Bazaar in Bishkek, Kyrgyzstan, since 2018. Before this work, stock, orders and sales were tracked by hand and in messenger chats, so the family could not see reliably what was on the shelf, what had been promised to customers or what was selling.

The system replaces these manual processes with one application. Customers browse a catalogue, mix any socks so that the price per pair falls with the size of the bag, ask for a better price in a chat, and pay online or when they collect the order. The owners and their staff manage products, stock, prices, deliveries, orders and wholesale requests in an administration interface. Every change of stock is recorded with its reason in a ledger, stock is locked during checkout so that the last pair cannot be sold twice, reservations that are not collected release their stock automatically, and the owners are alerted by email when a product runs low.

The central technical question is how a language model can be allowed to negotiate prices without being trusted with money. The design separates talking from deciding: the assistant, either a rule-based negotiator or a large language model, only proposes a discount, and a small deterministic component, the `PolicyValidator`, limits every proposal to a value the owners set for each product, which the model never sees. The check is repeated at checkout and enforced by the database, so a manipulated model cannot change what a customer pays.

The system is a progressive web application in English, Russian and Kyrgyz (React, TypeScript) backed by a Spring Boot REST API and a PostgreSQL database. It is verified by more than 500 automated tests, and an evaluation with sixteen simulated customers shows that neither assistant sold below the owners' limit.

Contents

1 Introduction **5**

1.1 Background 5

1.2 Related work 6

1.3 Motivation 7

1.4 Thesis objective 8

1.5 Research questions 9

1.6 Scope 9

1.7 Structure of the thesis 10

2 Specification **11**

2.1 Software perspective 11

2.2 User classes and characteristics 12

2.3 Use cases 12

2.3.1 Representative scenarios 14

2.4 Functional requirements 15

2.5 Non-functional requirements 17

2.6 Constraints 18

2.7 Assumptions and dependencies 19

2.8 User interface plan 19

3 User documentation **20**

3.1 System requirements 20

3.1.1 Server 20

3.1.2 Client 21

3.2 Installation and start 21

3.2.1 Demo accounts 21

3.2.2 Configuration 22

3.3 The shop 22

3.3.1 Everywhere: the header, languages, currencies and the footer . 22
3.3.2 The home page . . . 23
3.3.3 The shelves: browsing and filtering . . . 24
3.3.4 A product . . . 26
3.3.5 Asking for a better price . . . 27
3.3.6 The fitting room, the size guide and the sock finder . . . 28
3.3.7 The family planner and gift boxes . . . 29
3.3.8 The bag, checkout and payment . . . 31
3.3.9 Orders and pickup . . . 33
3.3.10 The account . . . 33
3.3.11 Wholesale and company gifts . . . 34
3.3.12 Finding the stall . . . 36
3.3.13 On a phone . . . 37
3.4 The administration . . . 38
3.4.1 For the whole team . . . 38
3.4.2 For the owners . . . 43
3.4.3 For the super admin . . . 45
3.5 Troubleshooting . . . 46
3.6 Limitations . . . 47

4 Developer documentation 48
4.1 Overview . . . 48
4.2 Technology stack . . . 50
4.3 Repository orientation . . . 50
4.4 System design . . . 51
4.4.1 Layers inside the backend . . . 51
4.4.2 Roles . . . 51
4.5 Data model . . . 52
4.5.1 Integrity in the database . . . 55
4.6 REST API . . . 55
4.7 Security . . . 57
4.8 Pricing and the PolicyValidator . . . 58
4.8.1 Collection pricing . . . 58
4.8.2 The validator . . . 58

4.9 The negotiation assistant . . . 61
4.9.1 Design . . . 61
4.9.2 Defences of the language-model negotiator . . . 63
4.10 Orders, stock and payment . . . 64
4.11 Further domain features . . . 66
4.12 Frontend . . . 68
4.12.1 Structure . . . 68
4.12.2 Design system . . . 69
4.12.3 State and data flow . . . 71
4.13 Running and building . . . 71
4.14 Design trade-offs and known limitations . . . 72

5 Testing and evaluation **73**
5.1 Testing strategy . . . 73
5.2 Backend tests . . . 74
5.3 Frontend tests . . . 76
5.4 The browser journey . . . 77
5.5 Phones . . . 77
5.6 Evaluation of the two assistants . . . 78
5.6.1 Method . . . 78
5.6.2 Results . . . 79
5.6.3 Discussion . . . 79
5.7 Requirements coverage . . . 80

6 Conclusions and future work **81**
6.1 Conclusion . . . 81
6.2 Answers to the research questions . . . 82
6.3 Future work . . . 83
6.4 Final remarks . . . 83

Acknowledgements **84**

Bibliography **84**

List of Figures **88**

List of Tables **90**

List of Codes **91**

Chapter 1

Introduction

1.1 Background

AK&VEN is a family brand of Korean-made socks. The name joins the first letters of my parents' names, **AK**ylbek and **VEN**era, with an ampersand drawn as the Kyrgyz ram's-horn ornament (*kochkor müyüz*) shaped as a heart. My family has traded at Dordoi Bazaar in Bishkek since the early 2000s and has sold socks there since 2018, from a shipping container numbered 70-E. Dordoi is one of the largest markets in Central Asia: thousands of containers stacked in long rows, where families shop for the season and traders from the regions of Kyrgyzstan, from Kazakhstan, Uzbekistan and Russia buy goods for their own shops.

Three habits shape how socks are sold there, and every one of them is missing from an ordinary online shop:

- **The price is agreed, not read.** A customer asks, the seller answers, and the price moves with the quantity, the season and the relationship. A fixed price tag can feel impersonal to these customers, and discount codes are unfamiliar to them.

- **People buy for the whole family.** One visit covers the father, the mother and the children, each in a different size; the larger the set, the lower the price per pair.

- **Shops buy by the case.** A case holds about 240 pairs in packs of 10 or 12, often mixed, and the price is given on request because it follows the dollar rate, the factory price and the delivery.

This thesis builds an integrated online shop and inventory system for the business, in which these habits can continue online: a customer can ask for a better price in a conversation, plan one order for the whole family, mix any socks to reach a lower price step, and collect the order at the container or have it delivered. Shops and companies can build a case and send a request that the family answers. Behind the shop, the same system keeps the stock, the orders and the sales of the business in one place.

1.2 Related work

E-commerce platforms. General platforms such as the open-source Shopizer [1], Medusa [2] and Saleor [3] offer catalogues, carts, orders and an administration interface. Their architecture was a model for this project in two respects: a single API serves both the shop and the administration, and the administration is only another client of that API with more rights. They assume fixed prices, however, with discounts expressed as coupons or price lists; none of them treats the price as the result of a conversation, and they are built for multi-tenant scale that a single family stall does not need.

Inventory management. Business systems such as Odoo [4] model stock as a sequence of moves between locations rather than as a single number, so that every quantity can be explained by the operations that produced it. The same idea appears in software architecture as event sourcing [5], in which every change is stored as an event and the current state is derived from them. AK&VEN adopts a lighter form suited to a single stall: the current stock is kept as a number for fast reads, and every change is also written to a ledger with its reason in the same database transaction. To prevent overselling, it relies on row-level locks in PostgreSQL [6], taken in a fixed order during checkout.

Negotiation dialogue. Automated negotiation has been studied in natural-language processing for years. Lewis et al. [7] trained end-to-end models to negotiate the division of items and found that they learned to deceive; He et al. [8] separated the negotiation *strategy* from the *generation* of the sentences, which made the agents' behaviour more controllable. This thesis follows the same separation in an engineering

setting: the strategy (how much may be given) is fixed by deterministic code and the owners' settings, and the language model is used for the conversation and a proposal only.

Reliability of language-model applications. Large language models follow instructions that arrive in their input, including instructions hidden in user text. Greshake et al. [9] showed that applications which let model output drive actions can be taken over by such prompt injection, and the OWASP Top 10 for LLM applications [10] lists prompt injection, sensitive information disclosure and excessive agency among the main risks. The common advice is to treat model output as untrusted input, to keep secrets out of the prompt and to limit what the model can cause. AK&VEN applies all three to money: the model never sees the limit or the cost price, its proposal is validated by code, and it cannot state a price in its own words.

Grounding. Retrieval-augmented generation [11] grounds model answers in retrieved documents rather than in the model's general knowledge. The assistant here receives the facts of the product being discussed (fabric, care, origin, pack size, stock, the public price ladder) selected for the question; the database already has a vector table (pgvector [12]) prepared for embedding-based retrieval in future work.

Progressive web applications. A progressive web application [13] is a website that can be installed like an app and can keep working offline with a service worker. For customers at a bazaar with an unreliable mobile connection this matters more than a native app store presence, and one application can serve customers and staff.

1.3 Motivation

The motivation is practical and personal. Until now the business has been run by hand. Stock is counted at the stall and kept in notebooks and in my parents' memory; orders arrive as messages on WhatsApp and Telegram; and how well a model sells is known only from experience. This causes three recurring problems. First, the recorded stock and the socks on the shelf drift apart, so a customer is sometimes promised a size that has already been sold. Second, orders are fulfilled late

because they are scattered across chats and depend on who is free to reply. Third, the family has no clear view of which products, sizes and customers bring the most sales, which makes reordering from the Korean suppliers a matter of guesswork.

An integrated system addresses all three: customers see the real assortment and stock, choose sizes from the shop's own size chart and place orders in one place; the family sees orders, stock, incoming goods and requests in one list, with every change of stock explained; and sales are summarised on a dashboard.

The difficulty is that bargaining is part of the brand, and a language model is the obvious tool for a conversation in three languages, but it is not safe to let a model decide prices. A shop that can be talked into a 90% discount loses money; a shop that refuses to talk is not Dordoi. The thesis is about resolving that tension with a design in which the model may talk freely while the price stays under the owners' control.

1.4 Thesis objective

The objective is to design, implement, test and evaluate a complete online shop for AK&VEN in which:

- customers can browse, choose sizes and colours, plan a family set, ask for a better price, pay and collect or receive their order, in English, Russian or Kyrgyz, on a phone, a tablet or a laptop;
- the owners and staff manage the catalogue, stock, prices, orders, requests and settings in an administration interface in the same three languages;
- stock stays correct under concurrent orders: every change is recorded with its reason, the last pair cannot be sold twice, reserved stock is released when it is not collected, and the owners are warned when a product runs low;
- no price below the owners' limit can ever reach an order, whatever the assistant says, and this guarantee is shown by automated tests and by an evaluation on simulated customers.

1.5 Research questions

RQ1. Can a language model hold a price negotiation for a real shop while a deterministic component guarantees that no discount above the owners' limit is ever applied?

RQ2. How does a language-model negotiator compare with a rule-based one on the same simulated customers, in terms of discounts given, sales made and how often the limit had to step in?

RQ3. Can the bazaar's habits (bargaining, family sets, price steps by quantity, wholesale by the case) be expressed in an online shop without making it harder to use than an ordinary one?

RQ4. Can the stock of a small shop stay consistent and explainable when online orders, reservations, cancellations, deliveries of new goods and manual corrections happen at the same time?

1.6 Scope

Built in this thesis: the shop and the administration as one progressive web application; the REST API with authentication and four roles; the catalogue with sections, cuts, colours, sizes, pictures and stock; inventory management with a stock ledger, incoming goods, reservations and low-stock alerts; collection pricing; the negotiation assistant with a rule-based and a language-model implementation; the bag and checkout with pickup at the stall, a courier and cargo carriers, payment through a *simulated* Apple Pay and Google Pay wallet or in person at handover; guest checkout; email notifications; a sales dashboard and order export; family profiles, gift boxes, reviews, saved products and wholesale requests; an audit log; the test suite; and deployment with Docker Compose and continuous integration.

Not built, on purpose: a real card payment provider (the wallet moves no money), SMS notifications, taxes, a payment rail through a Kyrgyz bank, demand forecasting and synchronisation with the family's messenger chats. These are discussed as future work in Chapter 6.

1.7 Structure of the thesis

Chapter 2 specifies the system: its users, use cases, and functional and non-functional requirements. Chapter 3 is the user documentation: how to install and start the shop, and how customers, staff, owners and the super admin use it. Chapter 4 is the developer documentation: the architecture, the data model, the API, security, the pricing and negotiation logic, and the design of the frontend. Chapter 5 describes how the system is tested and presents the evaluation of the two assistants. Chapter 6 answers the research questions and lists future work.

Chapter 2

Specification

This chapter defines what the system must do. It was written first as the requirement analysis of the first milestone and extended as the project grew; the requirements below are the ones the final program implements. Chapter 5 shows, for each of them, the automated test that demonstrates it.

2.1 Software perspective

AK&VEN is a new, self-contained web system: one server application with its database, and one web application that runs in the browser and can be installed on a phone or a computer. It does not replace the stall; it extends it. The stall remains the place where orders are collected and where the stock is kept, and the shop's data (the address of the stall, the opening hours, the contacts, the prices and the size chart) is entered by the owners in the administration rather than written into the code.

2.2 User classes and characteristics

User	Description
Visitor	Anyone who opens the shop. Browses the catalogue, uses the size guide, the fitting room and the family planner, fills the bag, and sends a wholesale request, all without an account.
Customer	A visitor with an account. Can also check out, ask the assistant for a better price, keep a family profile, saved socks and contact details, follow orders, write reviews and wait for sold-out options.
Staff	An employee of the stall. Manages products, stock and orders, hands orders over at the stall, moderates reviews and answers wholesale requests; cannot change prices or the owners' limits.
Owner (admin)	Akylbek and Venera. Everything staff can do, plus the limits per product, the price ladder, the exchange rates, delivery areas, contacts, opening hours, the size chart and customer accounts.
Super admin	The person who runs the installation. Everything an owner can do, plus the team's accounts, the choice of assistant and the audit log.
Assistant	Not a person: the negotiator called by the chat, either rule-based or a language model. Its behaviour and limits are part of the specification, not only of the implementation.
Payment provider	The wallet (Apple Pay or Google Pay) that turns a card into a one-time token. Simulated in this thesis.

Table 2.1: Users of the system

The customers are families and traders from Kyrgyzstan and the neighbouring countries. Many of them shop on a phone, often with a weak connection, and read Russian or Kyrgyz more comfortably than English. The owners are not technical users: the administration must use plain words, their language and few steps.

2.3 Use cases

Figures 2.1 and 2.2 show the use cases of the customer and of the shop team. The most important one, *Negotiate price*, involves the assistant as a secondary actor: the customer talks, the assistant proposes, and the system decides.

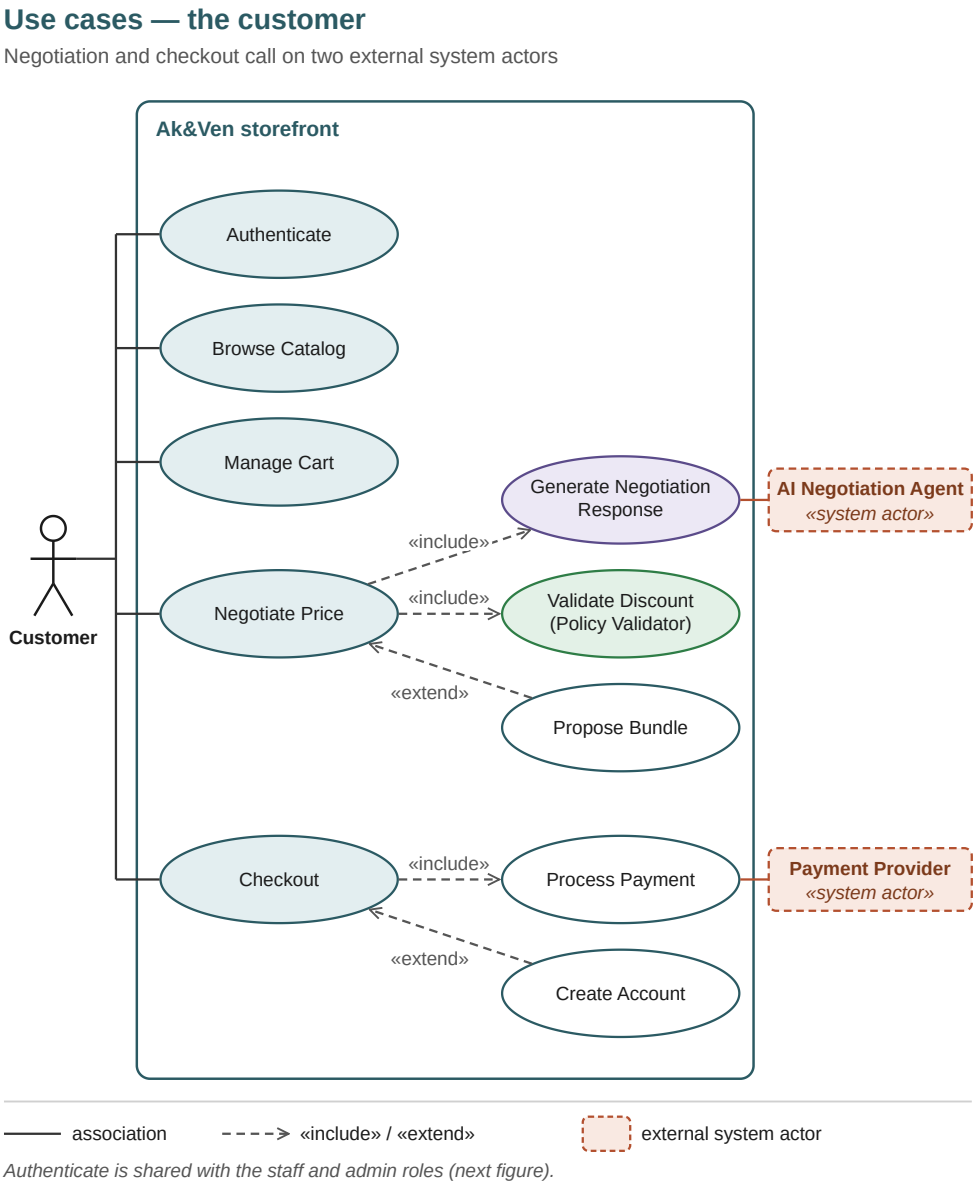


Figure 2.1: Use cases of visitors and customers

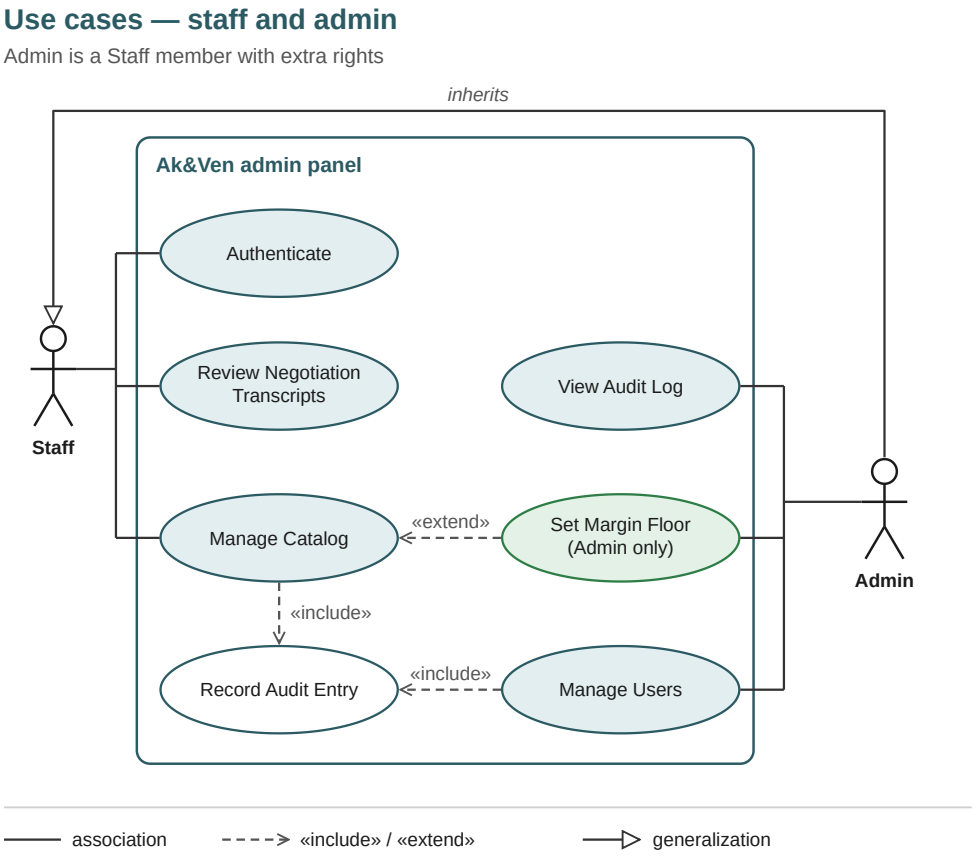


Figure 2.2: Use cases of the staff, the owners and the super admin

2.3.1 Representative scenarios

A family set. Aida needs socks for her husband (shoe size 43), herself (38) and her son (32). In the family planner she adds the three of them; the shop works out the sock size of each from its own size chart and shows the price step reached by the 15 pairs. She opens the shop with these sizes, fills the bag, pays with her phone's wallet and receives a six-digit pickup code. At the stall she says the code and the last four digits of her phone number, and Venera hands the bag over.

Bargaining. A customer opens a sock and writes "Can you do 50% off for 10 pairs?". The assistant answers in the customer's language with an offer. Whatever it proposes, the offer shown is at most the limit the owners set for that sock; it is kept for 24 hours, applies only to at least the negotiated number of pairs, and can be used once.

A shop from Osh. A shop owner builds a case of 240 pairs from the shelves of men's, women's and children's socks, writes the shop's name and a WhatsApp number, and sends the request. The shop answers with a reference number; the request appears in the family's inbox, where they reply with one tap in the customer's own messenger.

2.4 Functional requirements

Table 2.2 lists the functional requirements. FR-1 to FR-14 come from the first milestone; FR-15 to FR-26 were added as the shop grew.

ID	Requirement
FR-1	List the products with their variants (size, colour, pack), price and stock, filterable by audience, section, cut, size, colour and stock.
FR-2	Let a visitor fill a bag without an account; the bag survives a reload and works offline.
FR-3	Accept a negotiation message about a product from a signed-in customer and return the assistant's answer, in the customer's language, as part of a conversation.
FR-4	Ground the assistant's answer in the product's real data (fabric, care, origin, pack, stock, price ladder), not in the model's general knowledge.
FR-5	Check every proposed discount with a deterministic validator against the product's limit before it can be shown, stored or charged; check it again at checkout.
FR-6	Record the proposed and the allowed discount of every negotiation.
FR-7	Take payment only through the wallet interface; never receive or store card numbers.
FR-8	Create a paid order only with a confirmed payment reference.
FR-9	Require authentication for staff and owner actions; staff cannot change prices or limits.
FR-10	Let the team create, edit, remove and restore products, variants, pictures and stock.
FR-11	Restrict cost price and limit changes to owners.

ID	Requirement
FR-12	Show the negotiations with both numbers, and a dashboard of offers, limits applied and sales.
FR-13	Record an audit entry (who, what, before, after) for every change to the catalogue, prices, orders, accounts and settings.
FR-14	Let the super admin manage the team's accounts and read the audit log.
FR-15	Price a bag as one collection: a minimum order and a ladder of discounts by the number of pairs in the whole bag, set by the owners.
FR-16	Offer pickup at the stall with a one-time code, a courier in Bishkek and the Chui region, and cargo carriers to the regions, KZ, UZ and RU (paid to the carrier on receipt where the owners say so), each with a fee, a free-from amount, a time and a note set by the owners.
FR-17	Let customers keep a family profile (names, roles, shoe sizes, birthdays) and derive the sock size from the shop's size chart.
FR-18	Let customers build a gift box (box, ribbon, card) that is packed at the stall.
FR-19	Accept reviews only from customers who received the product, and publish them after approval.
FR-20	Let customers wait for a sold-out option and show the owners who is waiting.
FR-21	Accept wholesale and company requests without an account, give a reference, and show them to the team in an inbox with statuses.
FR-22	Remember a customer's contact and delivery details on request, fill them in at checkout, and forget them on request.
FR-23	Show the shop, the administration and all messages in English, Russian and Kyrgyz, with prices in US dollars, Kyrgyz som or Russian roubles at rates the owners set.
FR-24	Show whether the stall is open now in Bishkek time, its hours and how to find it.
FR-25	Let the super admin switch the assistant between the rules and the language model at run time, try it, and repeat the evaluation.
FR-26	Export the orders of a period as a spreadsheet file for the owners.

ID	Requirement
FR-27	Let customers order without an account, with an email address, and open, cancel and find their order again through a private link.
FR-28	Email customers in their language when an order is confirmed, handed over and cancelled, and when a wholesale request is received; keep every email readable by the team, and keep the shop working without a mail server.
FR-29	Record every change of stock (opening stock, goods received, sale, cancellation, manual correction) with its reason, quantity and author; let the team record arriving goods and read the history of each product variant.
FR-30	Never sell more pairs than are on the shelf, also when several customers check out at the same time; show stock on the way with its expected date, and email the owners when a sale leaves a product at the “few left” level.
FR-31	Let a customer reserve an order without paying online and pay at the stall or to the courier; hold the reserved stock for 48 hours and release it automatically when the order is not collected.

Table 2.2: Functional requirements

2.5 Non-functional requirements

ID	Category	Requirement
NFR-1	Security	Passwords are stored only as BCrypt hashes.
NFR-2	Security	Every endpoint enforces the roles; there is one API, not a separate administration backend.
NFR-3	Security	Customer text sent to the assistant is untrusted; no discount reaches an order without the validator.
NFR-4	Security	No card data is received, stored or logged.
NFR-5	Reliability	The bag works offline; prices are always recomputed by the server.

ID	Category	Requirement
NFR-6	Performance	An assistant answer arrives within a conversational time (target: under about 5 seconds).
NFR-7	Maintainability	Every change by the team can be reconstructed from the audit log alone.
NFR-8	Testability	The validator and the pricing are testable without the language model; the build fails under 80% line coverage.
NFR-9	Portability	One installable PWA serves customers and the team through role-gated routes.
NFR-10	Operability	Build and tests run automatically in CI on every push.
NFR-11	Usability	The shop works on phones, tablets and laptops; text meets WCAG 2.2 AA contrast; pages pass automated accessibility rules.
NFR-12	Integrity	The most important rules (stock never below zero, an allowed discount never above its proposal, a paid order has a payment reference) are also enforced by database constraints.

Table 2.3: Non-functional requirements

2.6 Constraints

- **Technology:** Java 17 and Spring Boot 3.5 for the server, PostgreSQL 16 with pgvector for the data, React 18 with TypeScript for the client, Docker Compose for running the whole system.
- **Time:** the ELTE milestones (requirements on 25 September, prototype on 20 October, final program on 10 November, draft on 22 November, submission on 1 December 2026).
- **Business:** there is no agreement with a Kyrgyz payment provider, so payment goes through a wallet interface with a simulated implementation; no real money moves.

- **Privacy:** real phone numbers, passwords and API keys are never written into the repository; they come from the environment of each installation.

2.7 Assumptions and dependencies

- A language model with an OpenAI-compatible chat API is available (Gemini by default [14]). Without a key, the rule-based assistant answers, so the shop never depends on the model.
- The demo catalogue (about 220 products with over 1,200 options) is drawn by the project itself until the family photographs the real products; the drawings use the same sock shapes and colours as the brand.
- Prices, limits, delivery fees and times in the demo are examples; the owners enter the real ones.

2.8 User interface plan

The interface was planned as wireframes for the five key screens (catalogue, product with the bag, checkout, the negotiation chat and the administration dashboard) on a phone, a tablet and a laptop before it was built; the final design keeps that structure. Its visual language comes from the stall itself (Section 4.12.2): kraft price tags, the family's handwriting, socks hanging on a washing line, and container 70-E.

Chapter 3

User documentation

This chapter explains how to run AK&VEN and how each kind of user works with it: the visitor and the customer in the shop (Section 3.3), and the staff, the owners and the super admin in the administration (Section 3.4). The screenshots show the demo catalogue in English; every screen is also available in Russian and Kyrgyz.

3.1 System requirements

3.1.1 Server

- Docker with Docker Compose [15] (on macOS, Docker Desktop or Colima; on Linux, Docker Engine);
- about 2 GB of free memory and 3 GB of disk space for the images and the database;
- for the optional language model: an API key of a provider with an OpenAI-compatible chat API (Gemini by default) and outbound internet access.

Without Docker, the backend needs Java 17 and Maven 3.9, the frontend Node.js 22, and the data PostgreSQL 16 with the pgvector extension; the developer documentation (Section 4.13) describes that setup.

3.1.2 Client

Any current browser (Chrome, Edge, Safari, Firefox) on a phone, tablet or computer. The shop can be installed as an app from the browser's menu ("Add to Home Screen" or "Install"); once installed it opens in its own window and the bag keeps working without a connection. JavaScript must be enabled.

3.2 Installation and start

From the project folder, one command builds and starts the database, the backend and the web server:

```
1 docker compose up --build
2 # then open http://localhost:8081 in a browser
```

Code 3.1: Starting the shop

The first build takes a few minutes. The backend has started when its log ends with `Successfully applied 45 migrations`; the database tables are created and, in the demo, filled with the catalogue. To start again from an empty database, run `docker compose down -v` and start again.

3.2.1 Demo accounts

Role	Email	Password
Super admin	`superadmin@akven.test`	`changeme-super`
Owner	`admin@akven.test`	`changeme-admin`
Staff	`staff@akven.test`	`changeme-staff`
Customer	create one with Sign in, then Create an account	

Table 3.1: Demo accounts

These accounts exist only in the demo. On a real installation the first super admin is created from the environment, and the demo accounts are switched off at every start.

3.2.2 Configuration

Settings are read from environment variables, usually from a file `.env` next to `docker-compose.yml` (an example is in `.env.example`). None of them is required for the demo.

Variable	Meaning
`SUPER_ADMIN_EMAIL`, `_PASSWORD`	The first super admin of a real installation; switches the demo accounts off.
`DEMO_ACCOUNTS=on`	Keeps the demo accounts for a presentation.
`LLM_API_KEY`, `LLM_MODEL`	Lets a language model answer the chat (Gemini by default).
`SHOP_CONTACTS`	The family's real phone numbers and messengers, kept out of the repository.
`JWT_SECRET`	Optional signing key for logins; without it each installation creates its own.
`CORS_ALLOWED_ORIGINS`	The web addresses allowed to call the API from a browser.

Table 3.2: The main settings

3.3 The shop

3.3.1 Everywhere: the header, languages, currencies and the footer

The header holds the shelves (Shop, Men, Women, Kids, Gift Sets, Our Story), the logo, a Wholesale button, and icons for the stall, search, saved socks, the account and the bag. The bar above it switches the currency (US dollars, Kyrgyz som, Russian roubles, converted at the owners' rates), the language (EN, RUS, KYR, the last two written in Cyrillic) and light or dark colours. The whole site, including messages from the server and the assistant's answers, follows the chosen language. On a phone the shelves move into a menu.

The footer (Figure 3.1) answers the last questions a visitor has: whether the stall is open right now (the sign on container 70-E reads open or closed in Bishkek time), the address with links to Google Maps and 2GIS, how pickup, delivery and payment work, every page grouped, the contacts, and a link to the privacy page, which explains in plain words what the shop keeps and why.



Figure 3.1: The footer: the stall's door with its open or closed sign, the promises as sewn labels, every page and the contacts

3.3.2 The home page

The home page (Figure 3.2) opens with one line about the shop, *Korean socks for the whole family.*, beside a still life of the everyday colours in an AK&VEN box on bojagi tissue and a felt runner, with the stall's round stamp and a red dojang seal. Two buttons lead to the socks and to the story, and three facts sit underneath: sizes from babies to adults, pickup at Dordoi or delivery, and a family-run stall since 2018. Below follow the reasons to order (a felt band), what the visitor looked at last, the four shelves as washing lines on their own fabrics (Figure 3.3), the bestsellers with their rank stamped in red, the price ladder as kraft tags on felt, what families say (each review a note on hanji paper with the writer's initial in a seal), the story of the two names, a short scene in which container 70-E opens as the visitor scrolls, and a way in for shops and companies. The fitting room, gift boxes, the family planner and the notes pinned at the stall have pages of their own, one tap away in the menu.

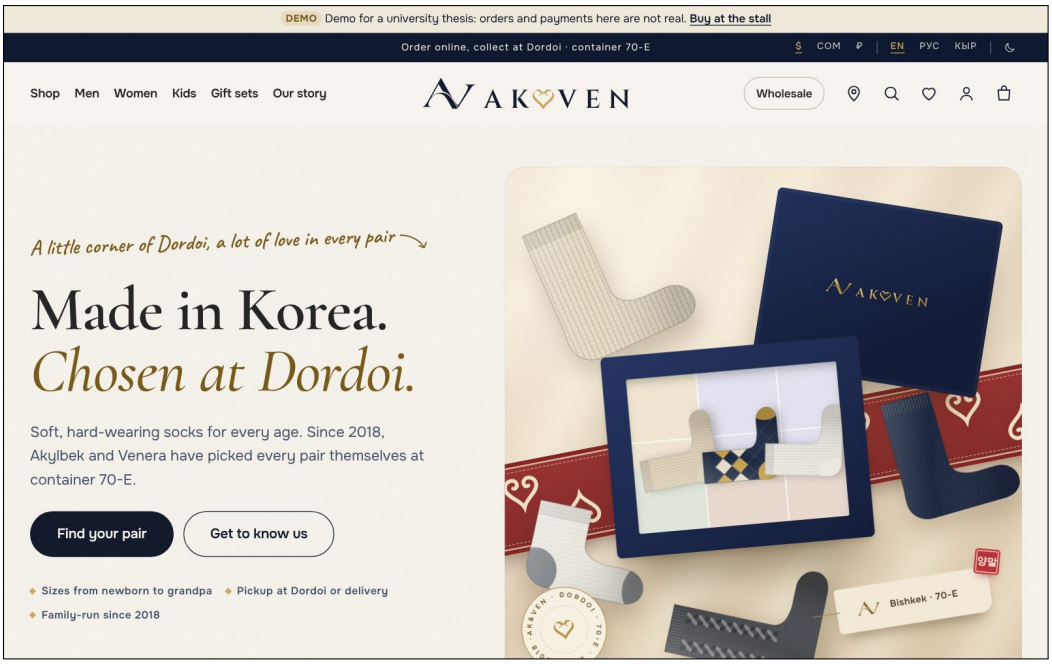


Figure 3.2: The home page

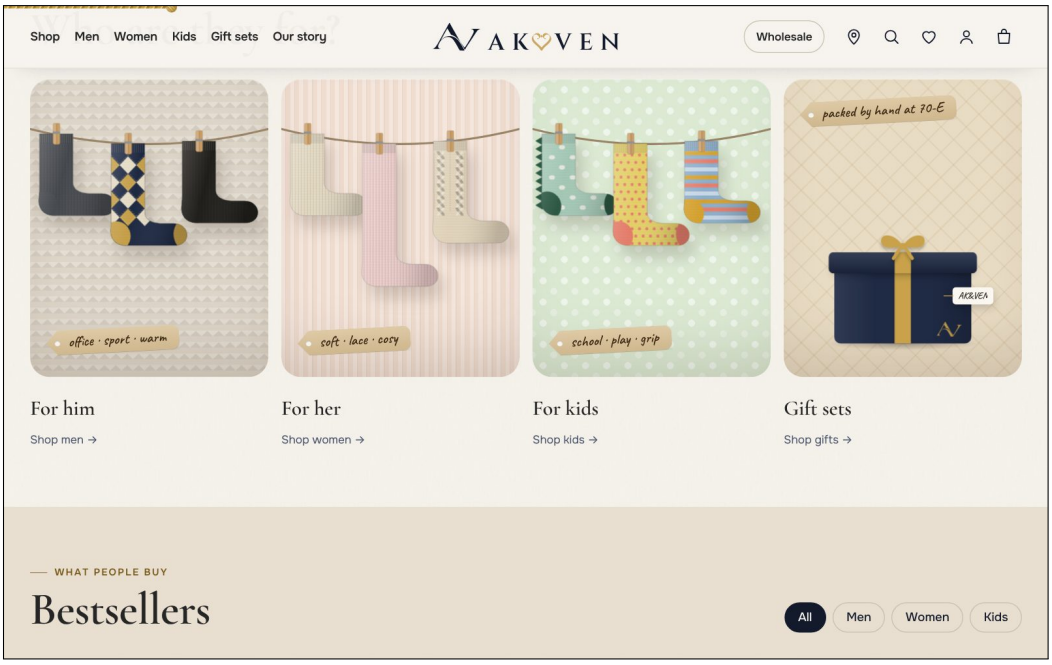


Figure 3.3: Who are they for? Each shelf on its own fabric, with its socks on the line

3.3.3 The shelves: browsing and filtering

A shelf (Figure 3.4) opens on its fabric with its socks pegged on the line and a tag with the number of products. The tabs switch between all socks, men, women, children and gift boxes. The owners' sections are drawn as occasions (Classic, Casual, Sport, Thermal, Home, Heritage); below them, a bar that stays in reach while

scrolling filters by height (no-show to knee-high), by size and by colour, shows only socks in stock and sorts by newest, name or price. Every number on a filter says how many products it would leave; a choice that would leave none is disabled. Each chosen filter appears as a tag that removes it with one tap (Figure 3.5). On a phone the filters fold behind one *Filters* button, which shows how many are chosen, so the socks start on the first screen. The list grows twelve at a time with *Show 12 more*, and a thread shows how much of the shelf has been seen. Everything is kept in the address, so a filtered shelf can be bookmarked, shared or reached again with the Back button.

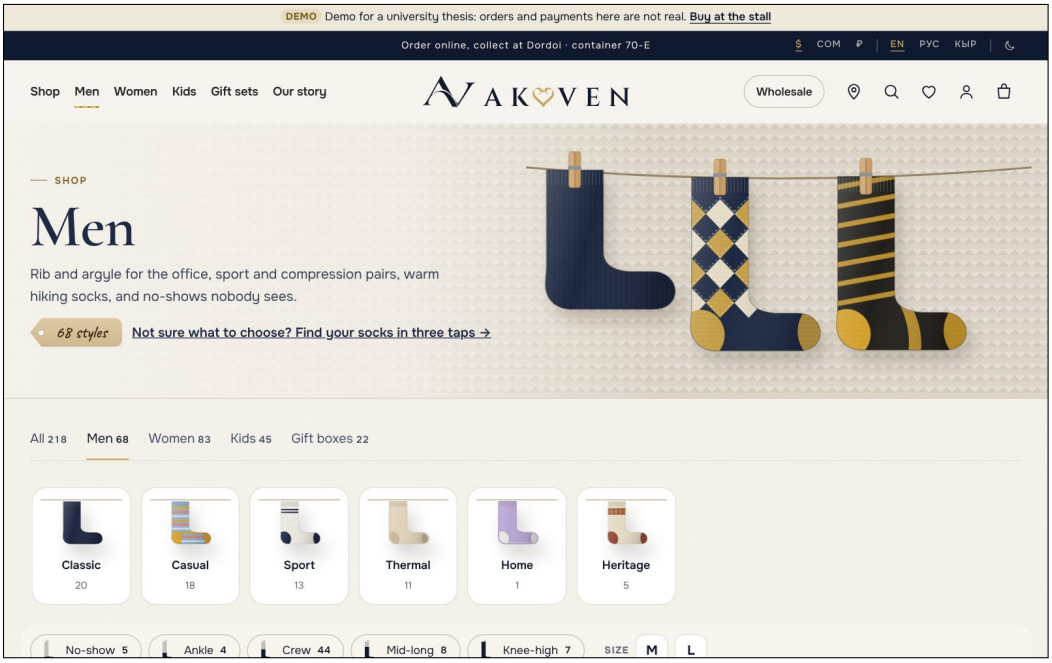


Figure 3.4: The men's shelf

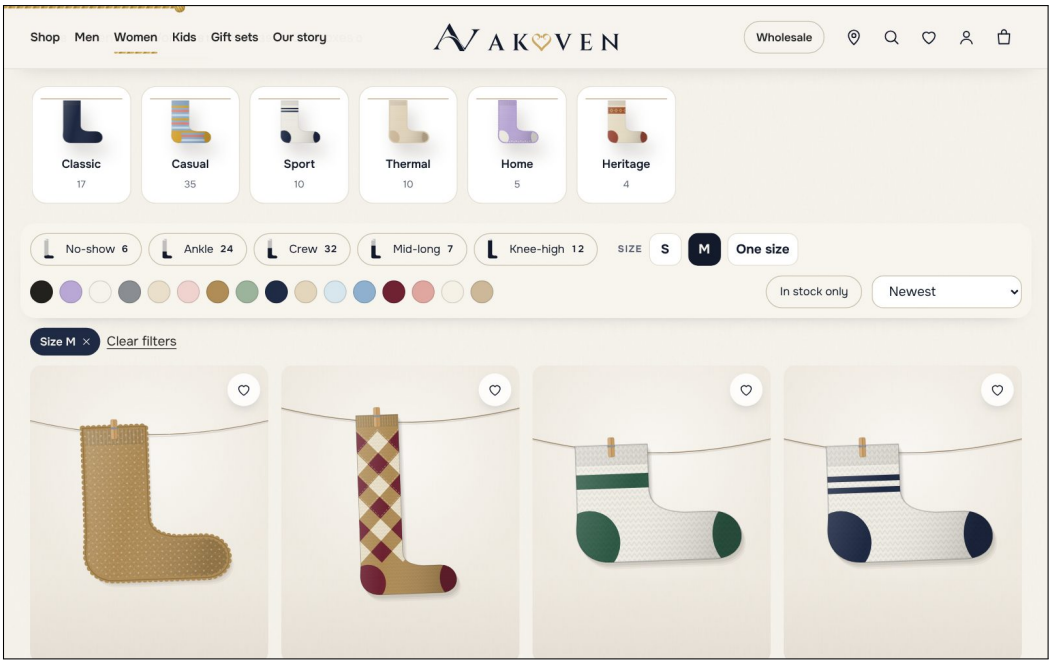


Figure 3.5: Occasions, heights, sizes and colours; the chosen size shown as a tag

On every card the heart saves the sock, and *Quick view* lets the visitor choose a colour and a size and add it to the bag without leaving the shelf.

3.3.4 A product

The product page (Figure 3.6) shows the pictures (the picture follows the chosen colour), the name, the price, the colours and sizes, the honest stock (in stock, only a few left, arriving in about n days, or sold out with *Tell me when it's back*), the quantity and *Add to bag*. Under them come when the order can be collected, the delivery countries and the collection discount, then *How the price falls* (Figure 3.7): a slider of pairs that shows the price of one pair and of all of them as the pairs add up, with the shop's price steps marked. Further down are the size finder, fabric, quality and care, the same design for the rest of the family, reviews and *Complete the set*. When the buy button scrolls out of view, a small bar with the picture, the price and *Add to bag* stays at the bottom of the screen.

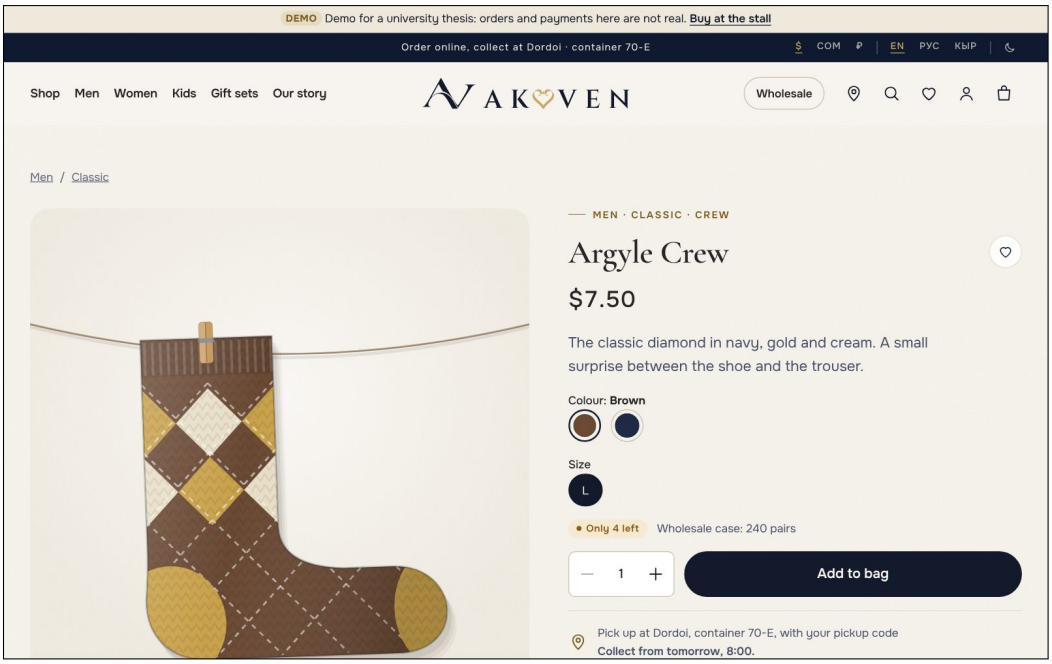


Figure 3.6: A product page

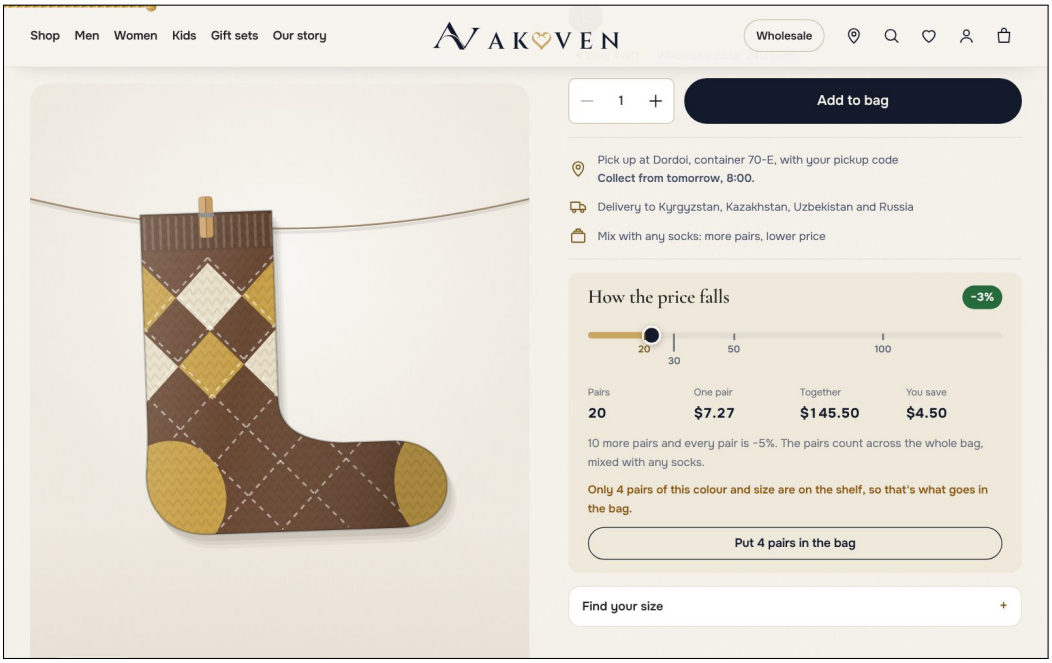


Figure 3.7: How the price falls: the price per pair and in total as the number of pairs grows

3.3.5 Asking for a better price

A signed-in customer can ask for a better price in the chat on the product page (Figure 3.8), in any of the three languages, in their own words: “Can you give me 50% off for 10 pairs?”. The assistant answers as the stall would, remembers the

conversation, answers questions about the sock, and shows an offer with the old and the new price. The offer can be put into the bag with one tap. The rules of an offer are shown with it:

- it never goes below the owners' limit for that sock, whatever was asked;
- it is valid for 24 hours, for at least the number of pairs it was made for, and can be used once;
- at checkout, the better of the offer and the collection discount applies, never both.

A customer can send twenty messages in ten minutes; after that the chat asks them to wait a little.

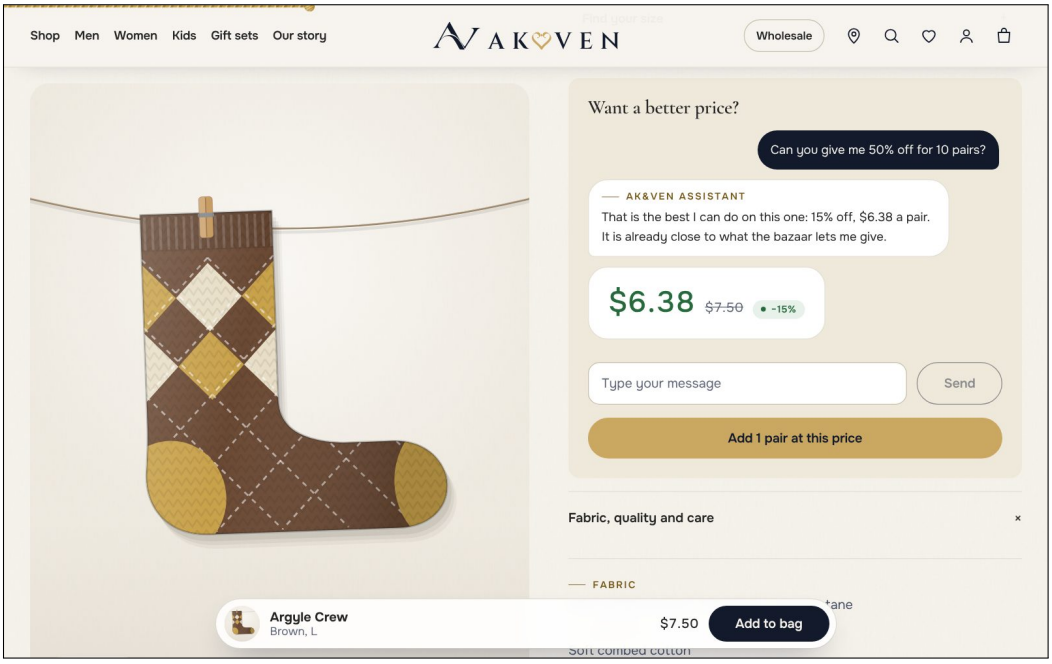


Figure 3.8: The chat: asked for 50%, the assistant offers what the owners allow

3.3.6 The fitting room, the size guide and the sock finder

The fitting room (Figure 3.9) draws the shop's own socks on a leg: the visitor chooses who is trying on, the shoes (sneakers, dress shoes, loafers, boots or at home) and a height; each height is rated for those shoes with a reason, the shoe size gives the sock size, and the trousers can be set to the visitor's colour (*Match my trousers* finds socks that go with them). The size guide gives the sizes each customer knows (EU and US in English, local sizes in Russian and Kyrgyz) and how to measure a

foot. The sock finder asks who, how high and what for in three taps and shows the matching socks.

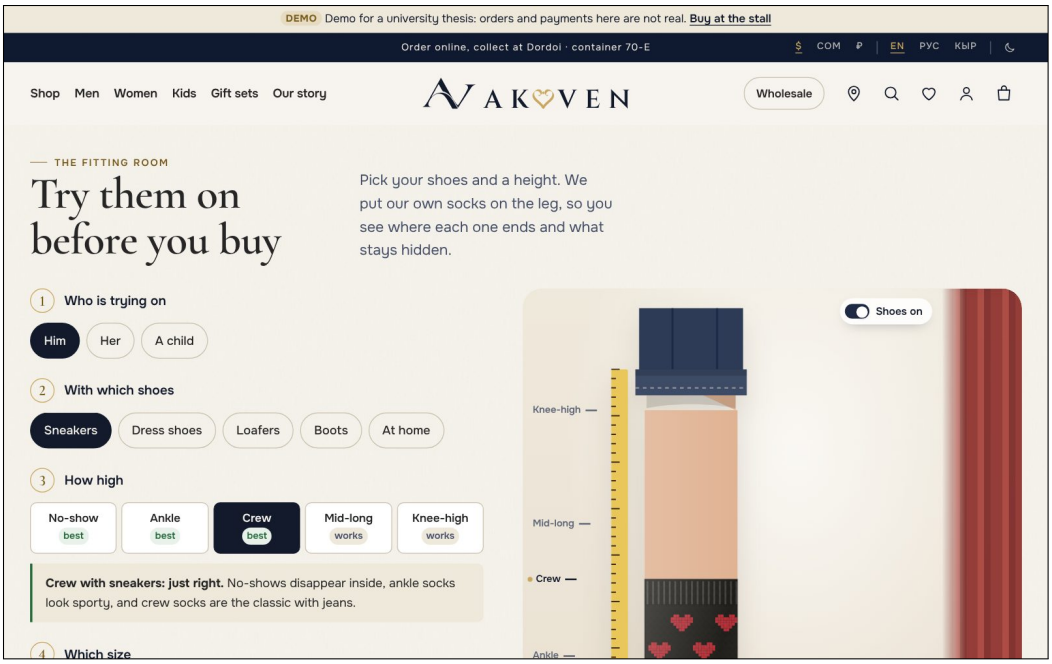


Figure 3.9: The fitting room

3.3.7 The family planner and gift boxes

In the family planner on the home page (Figure 3.10) a customer adds each person and their shoe size and the number of pairs. The sock size comes from the size chart, and the price step reached by the whole set is shown. The list can be taken to the shop with these sizes or sent to the family in a message. Signed in, the family (names, sizes, birthdays) is kept in the account; product pages then say who in the family a shelf fits, and a birthday within a month suggests a gift box.

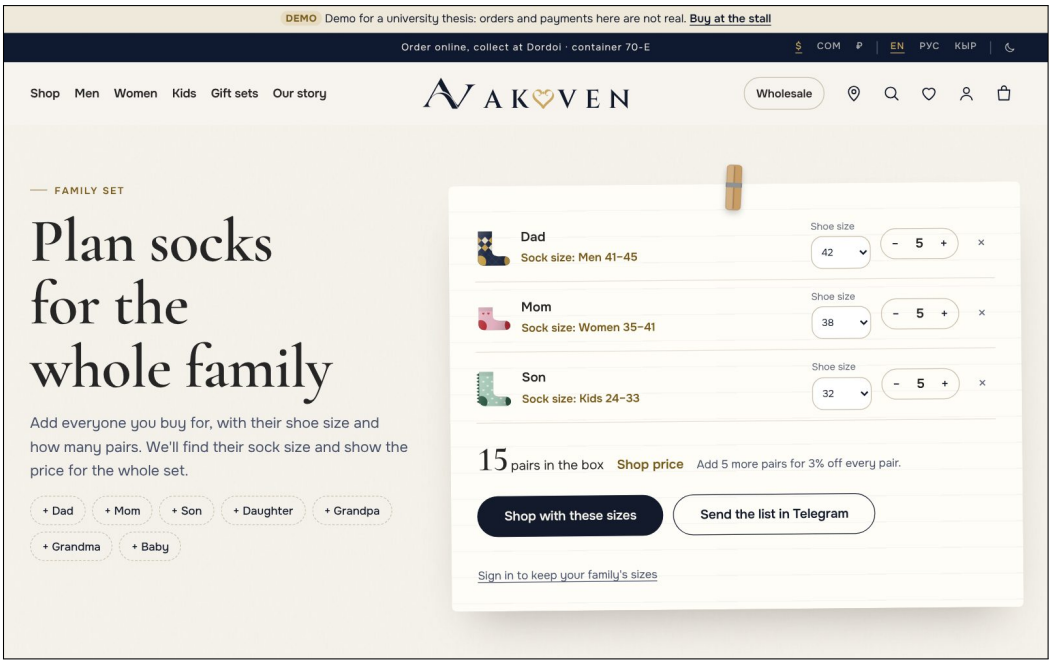


Figure 3.10: The family planner, kept on a sheet of paper pegged to the page

A gift box (Figure 3.11) is built in three steps: a kraft, navy or red box with a ribbon, a hand-written card, and socks from any shelf. The drawing shows the socks in the box and the lid closing over them; the family packs it at the stall, and the order tells them how.

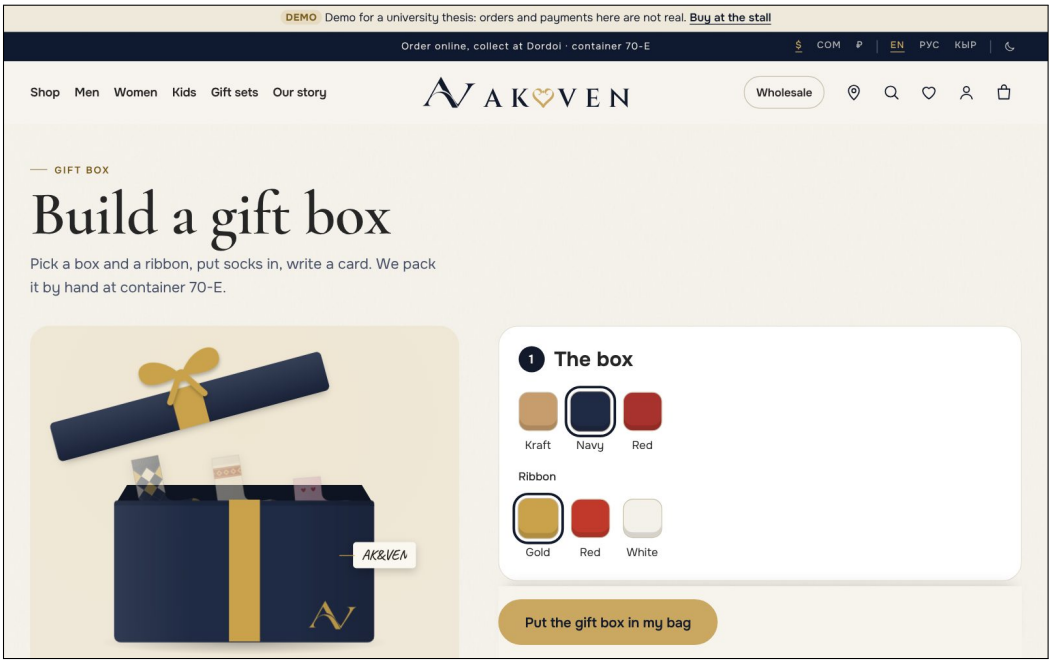


Figure 3.11: Building a gift box

3.3.8 The bag, checkout and payment

The bag (Figure 3.12) counts all its pairs as one collection: it shows the minimum order, the step reached and how many more pairs reach the next one. Prices are always taken from the server; if a price or the stock changed since the sock was added, the bag says so. A line can be removed or saved for later, which moves it to the saved socks.

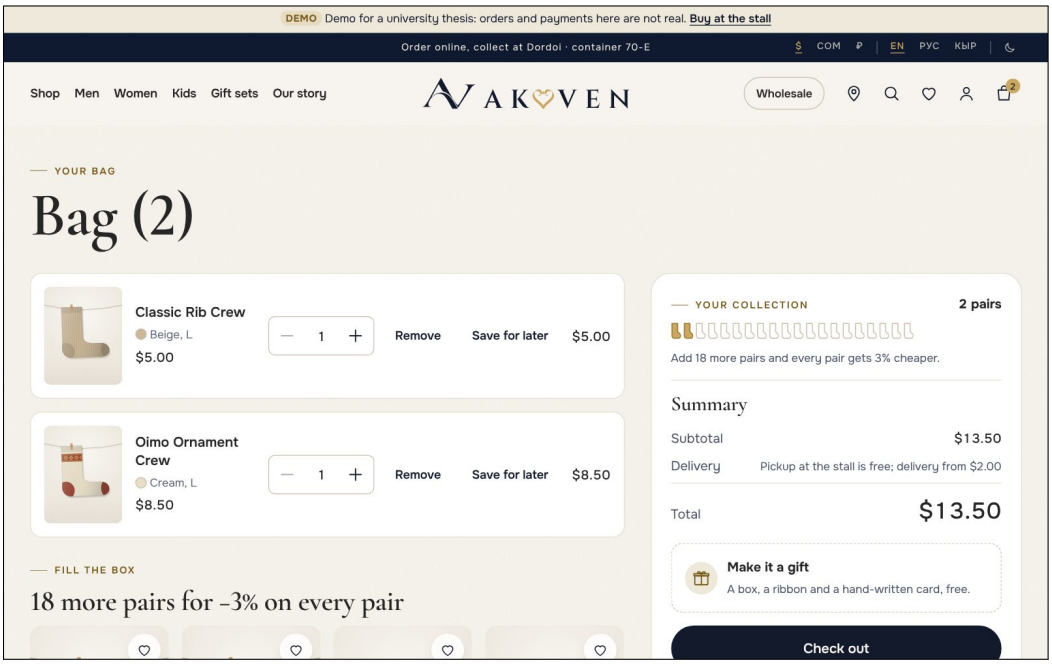


Figure 3.12: The bag as one collection

Checkout (Figure 3.13) needs no account: a guest gives an email address, where the confirmation and a private link to the order go; signing in adds saved details, agreed prices and the list of orders. It asks for a name and a phone number and how to receive the order: pickup at container 70-E, a courier in Bishkek and the Chui region, or cargo to the regions, Kazakhstan, Uzbekistan or Russia. Each area or carrier shows its fee, the days it takes, from which amount delivery is free and the owners' note; a carrier paid on receipt says so, and the summary says how much more makes delivery free. *Remember my details* keeps the name, phone, area and address for next time. The customer then either reserves the order and pays in person, or pays online. *Reserve, pay at the stall* (*Order, pay on delivery* for a delivery) charges nothing: the socks are put aside for 48 hours and paid in cash or with a bank app at the stall or to the courier; if nobody comes, the reservation ends by itself, the socks go back on the shelf and the customer gets an email. Online payment is with Apple

Pay or Google Pay (Figure 3.14); the wallet in this thesis is simulated and the dialog says so, and it can be told to decline in order to show what happens then: nothing is charged and the bag stays. While payments are simulated, the top bar of every page says that online card payment is coming soon and that socks can be reserved and paid for at the stall.

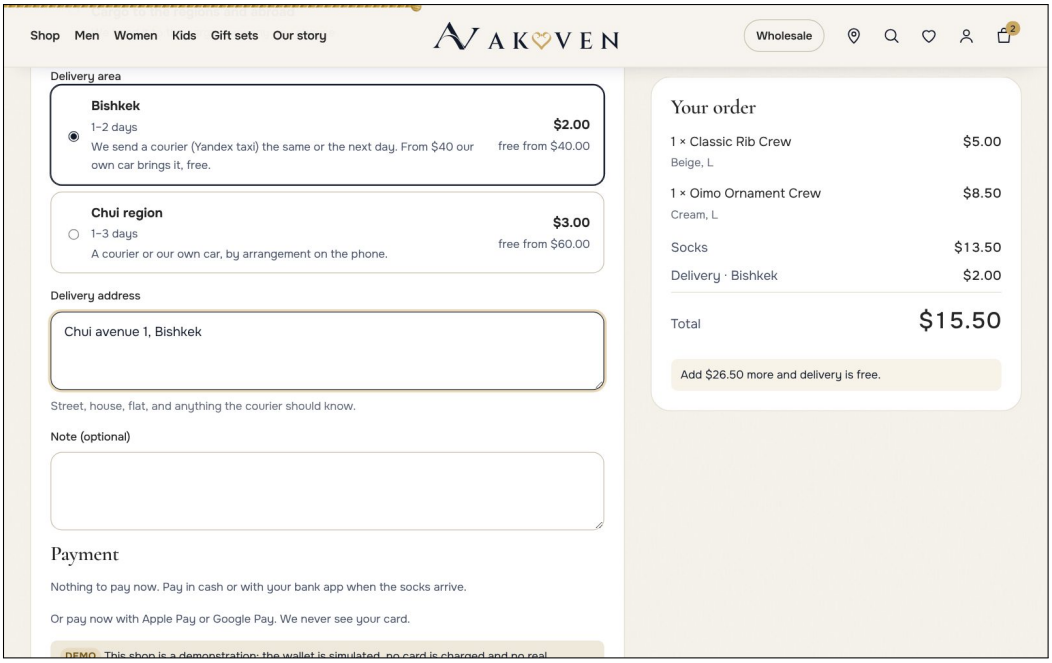


Figure 3.13: Checkout with a delivery area

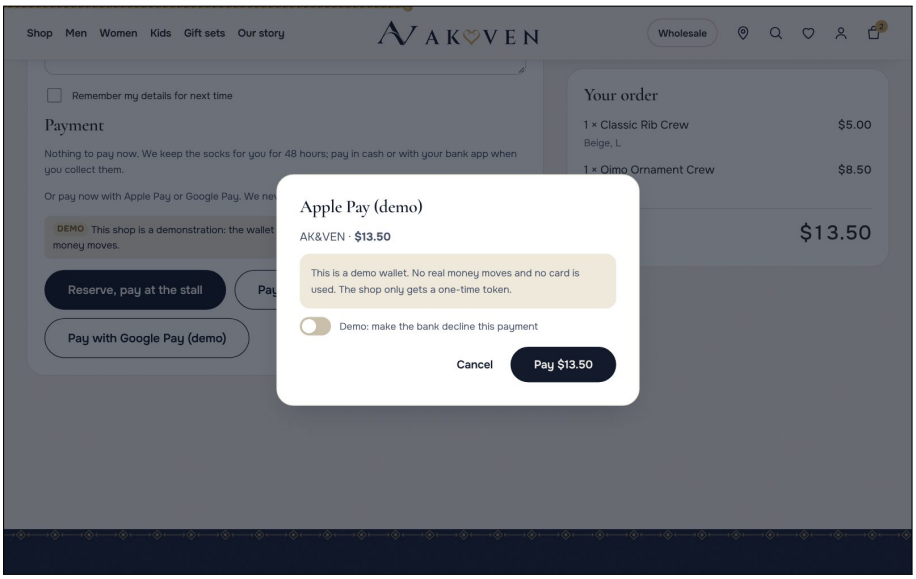


Figure 3.14: The simulated wallet

3.3.9 Orders and pickup

A paid order (Figure 3.15) shows its timeline (placed, paid, ready or on its way, delivered), the socks, the delivery area and time, and for pickup a six-digit code and a QR code. At the stall the customer says the code and the last four digits of their phone number. A reserved order shows until when it is kept and the same pickup code; it is paid at handover. A paid or reserved order can be cancelled until it is handed over; a payment is refunded, and the socks go back on the shelf. *Order these again* fills the bag with the same socks. Each step is also emailed in the customer's language. A guest's order lives at its private link; *Find my order* emails a lost link again, given the email address and the order number.

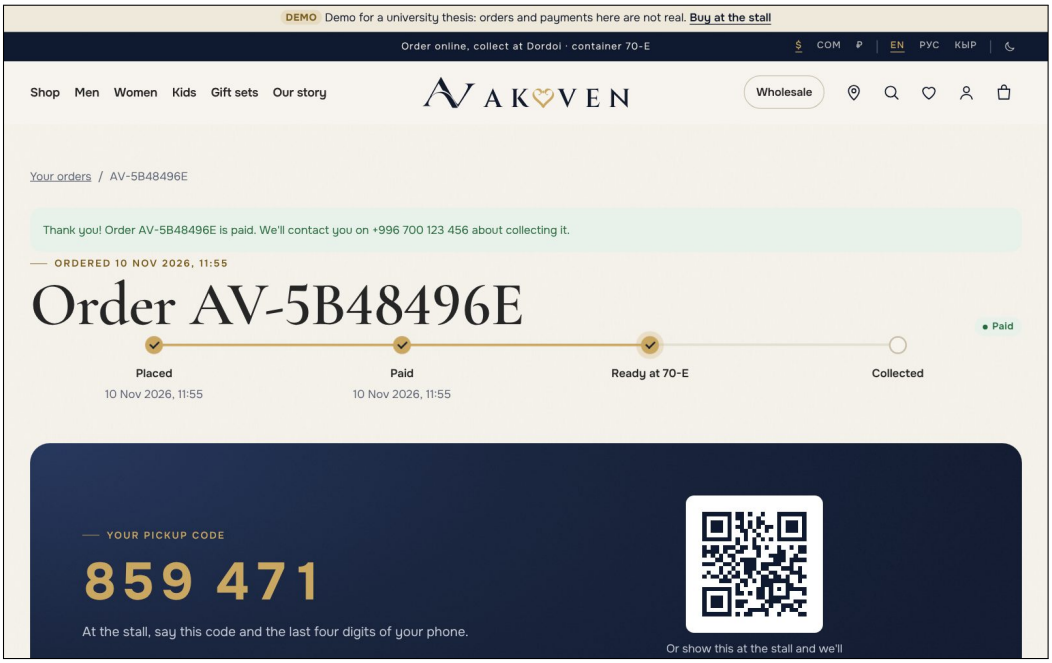


Figure 3.15: A paid order

3.3.10 The account

The account (Figure 3.16) holds the family, the saved contact and delivery details (to change or forget), the sold-out options the customer waits for, and the recent orders. Saved socks (the heart) are kept in the browser and, once signed in, with the account; the list can be shared as a link. Socks the customer looked at last appear as a row of pegged photos under the shelves and on product pages, and can be cleared.

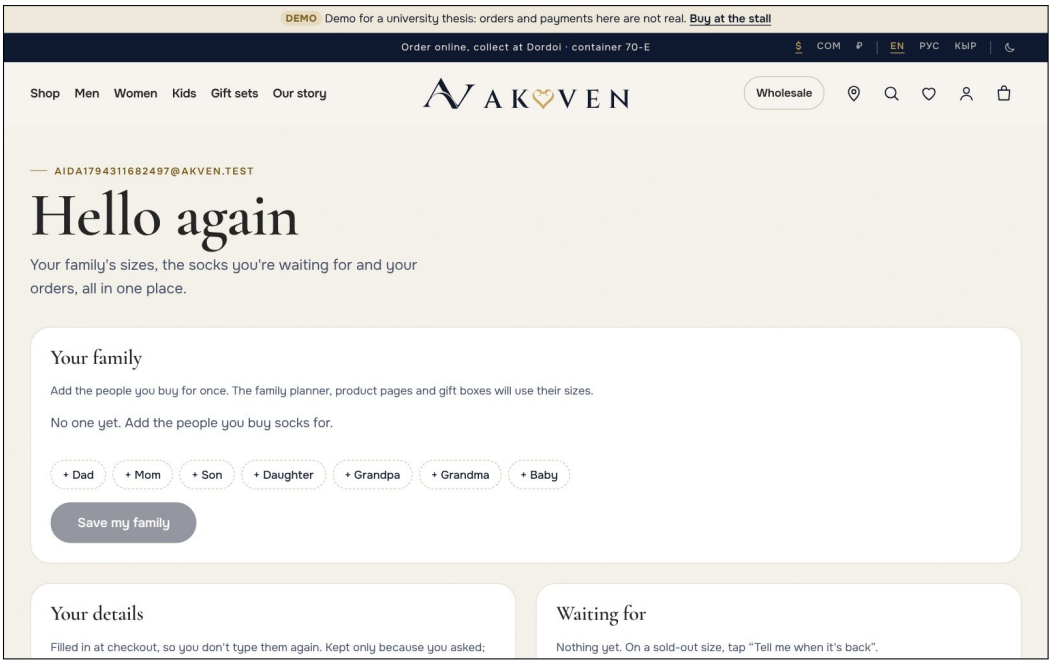


Figure 3.16: The account

3.3.11 Wholesale and company gifts

The wholesale page (Figure 3.17) is for shops, market stalls, teams and companies. It explains who orders, how a case is made, and the facts a buyer needs (packs of 10 to 12, about 240 pairs to a case, the four countries). In *Build a case* (Figure 3.18) the buyer works from the real shelves: search or browse a shelf, open a sock, and choose dozens of each colour and size with today's stock beside it (with a warning when the case asks for more than there is). The case, drawn as a cardboard carton with the stall's stamp, fills as pairs are added and shows the price step it reaches and an estimate; its exact list, one line per colour and size with the product code, goes into the request. The request (Figure 3.19) needs the name of the shop, a city and a way to answer (WhatsApp, Telegram, a call or e-mail). It is kept by the shop and answered with a reference number such as W-5018A33E, and the same text can be sent to the family's messenger. Wholesale prices are not published, because they follow the dollar rate, the factory price and the delivery; the family answers with a price.

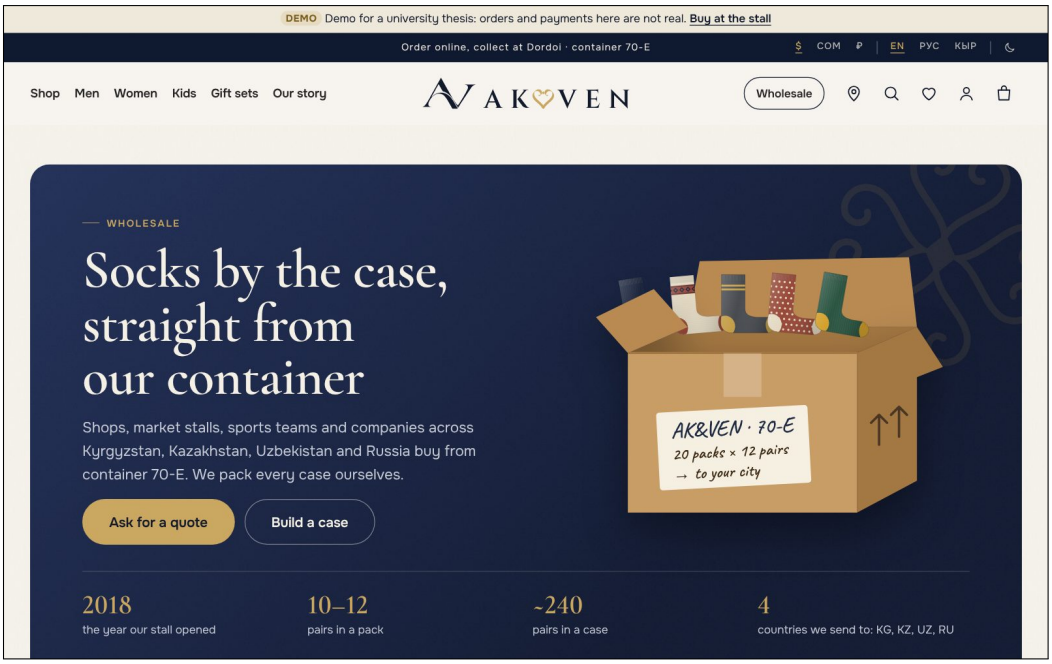


Figure 3.17: The wholesale page

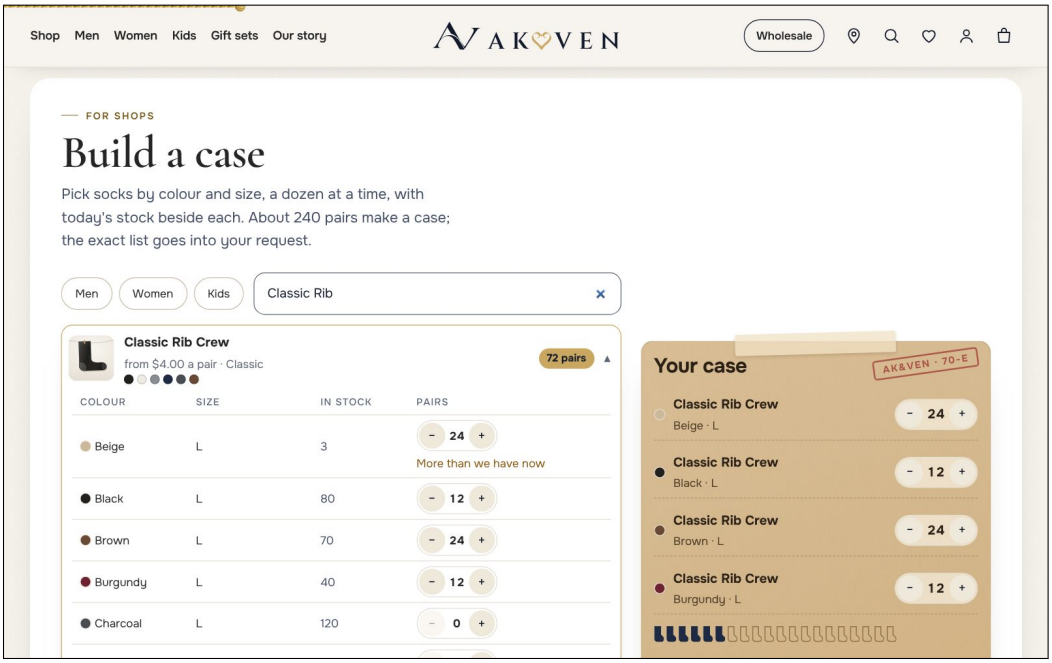


Figure 3.18: Building a case

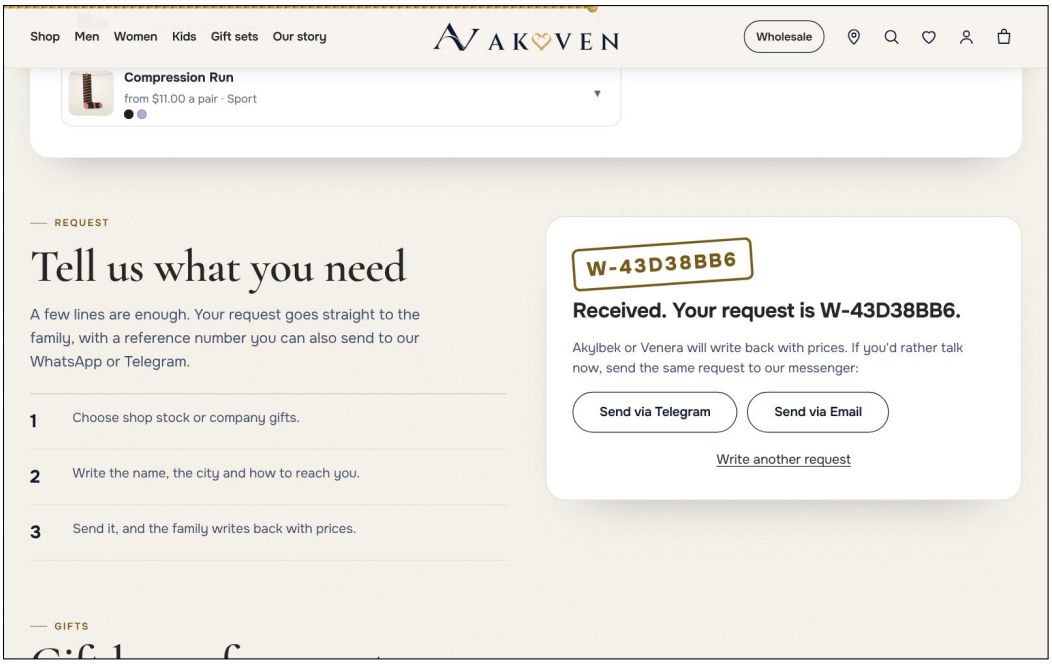


Figure 3.19: A request sent, with its reference

3.3.12 Finding the stall

Find us at Dordoi (Figure 3.20) tells whether the stall is open now, the week's hours, the address step by step inside the bazaar, and links to Google Maps and 2GIS. The address can be copied in one tap. Two further parts answer the questions people ask most: how collecting an order works (three short steps, as kraft tags), and where the shop delivers, read live from the delivery areas the owners set in the administration, each with its way (courier or cargo), days, fee and the amount above which it is free, or "paid to the carrier".

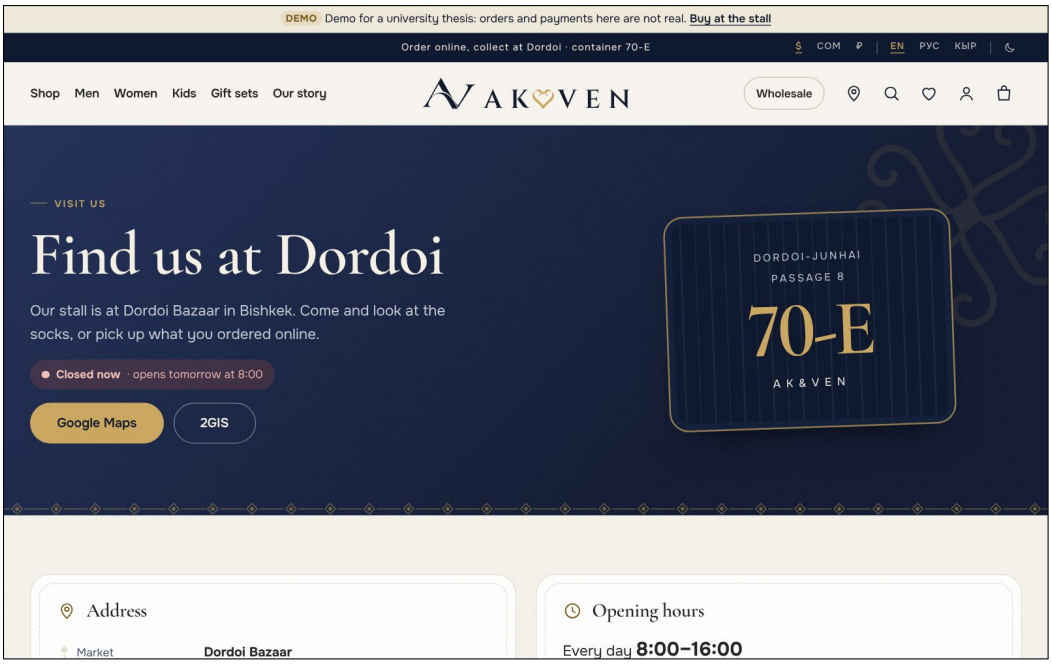


Figure 3.20: Find us at Dordoi

3.3.13 On a phone

Every page is laid out for phones, tablets and laptops. Figure 3.21 shows the home page and the children's shelf on a phone in Kyrgyz.

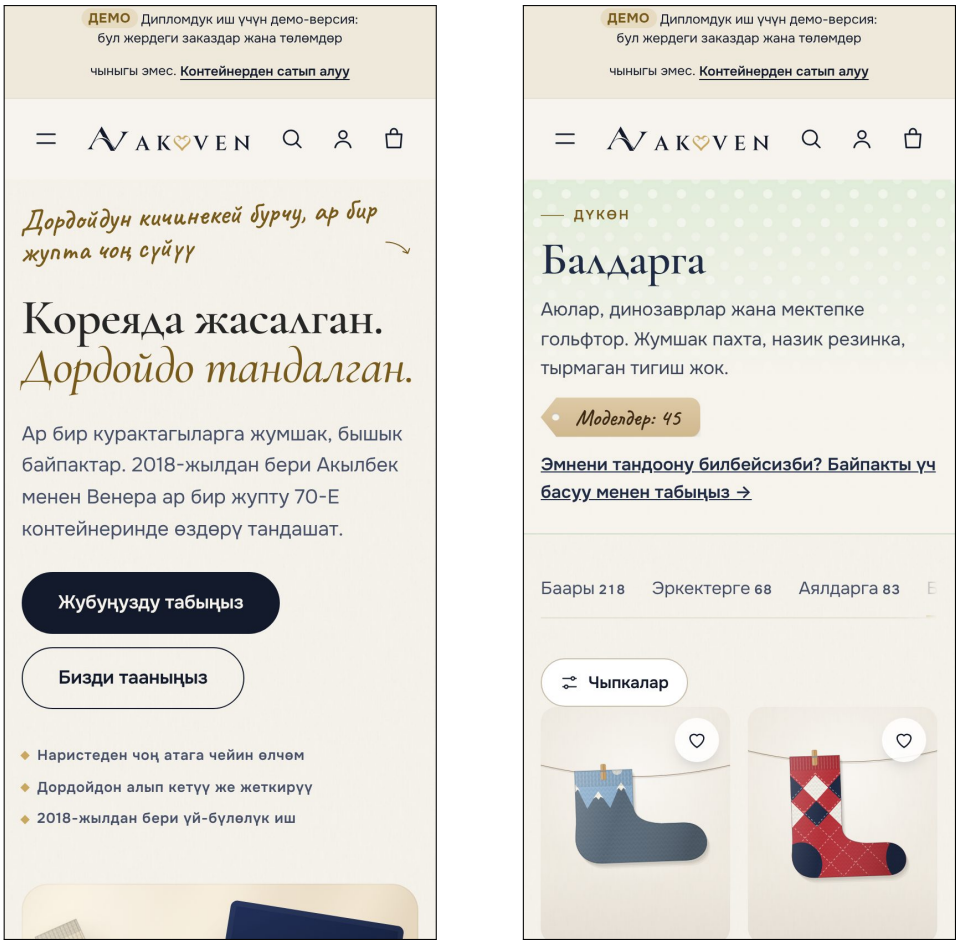


Figure 3.21: The home page and the children's shelf on a phone, in Kyrgyz

3.4 The administration

The administration is opened at `/admin` by a staff, owner or super admin account. Its menu (on the left) groups the screens by who may use them: *Shop* for everyone in the team, *Owners* for the owners and *System* for the super admin. The language can be switched at the bottom of the menu.

3.4.1 For the whole team

Products. The list (Figure 3.22) can be searched and filtered (in the shop, removed). One screen (Figure 3.23) adds or edits a product: name, audience, section, cut, description, fabric, quality, care and origin; its options (colour with swatch, size, pack, price, stock, stock on the way and when it arrives); and its pictures, each of which can belong to one colour. *Goods arrived* adds a delivery to an option's stock (and takes it off "on the way"), and *Stock history* (Figure 3.24) lists every change to

the stock with its reason: the opening count, a delivery, a sale linked to its order, a cancelled sale, or a count corrected by hand. Pictures are uploaded as JPEG, PNG or WebP of up to 5 MB. Removing a product hides it from the shop; it can be restored. Every change appears in the product's history.

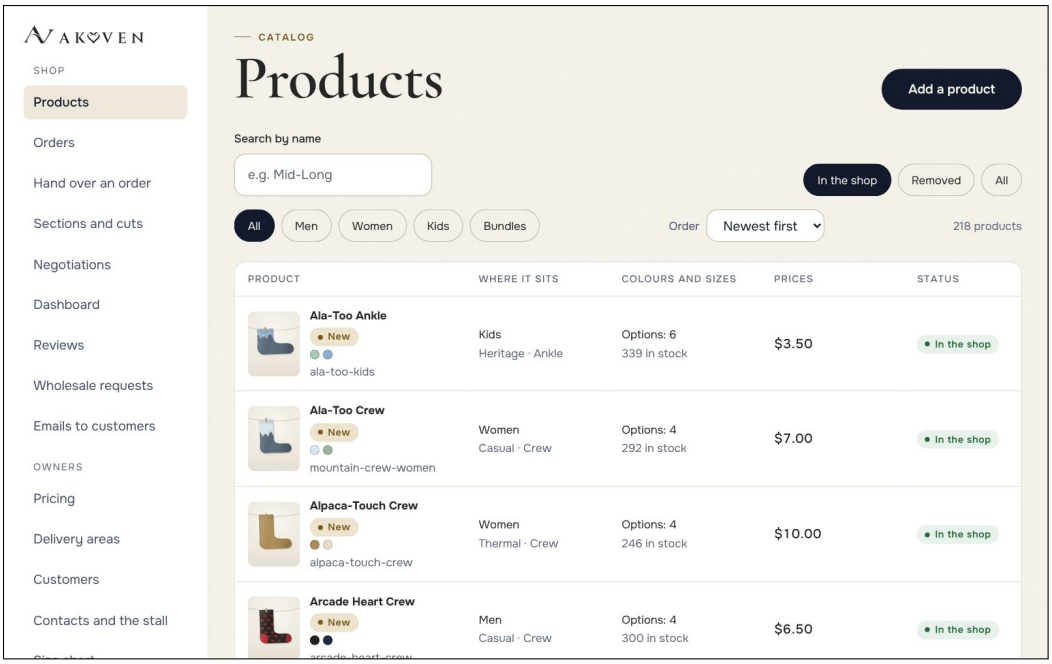


Figure 3.22: Products

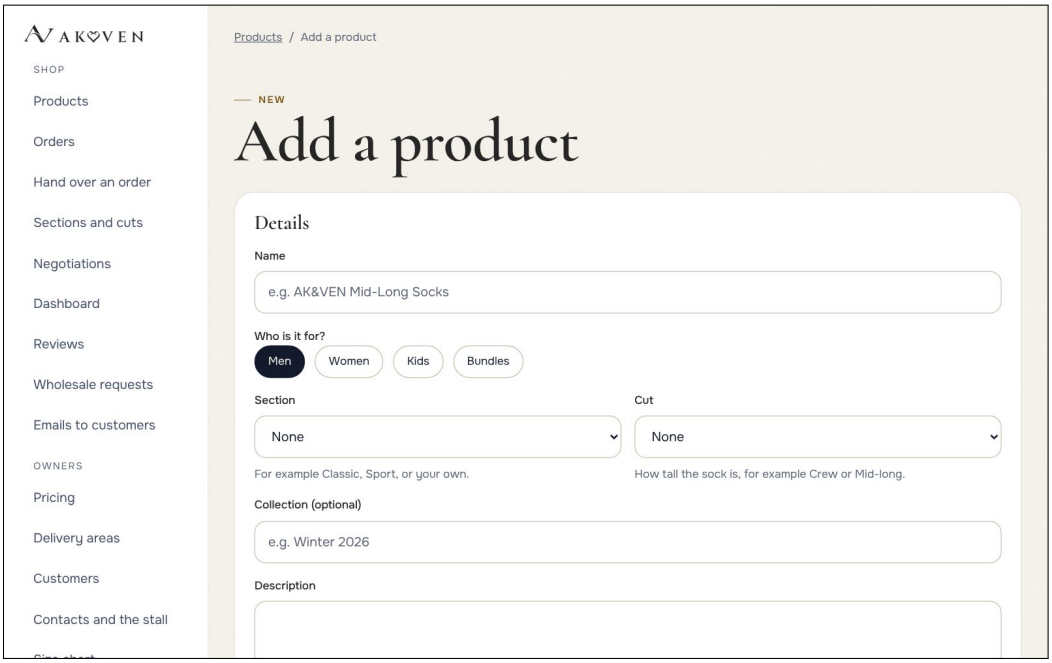


Figure 3.23: Adding a product

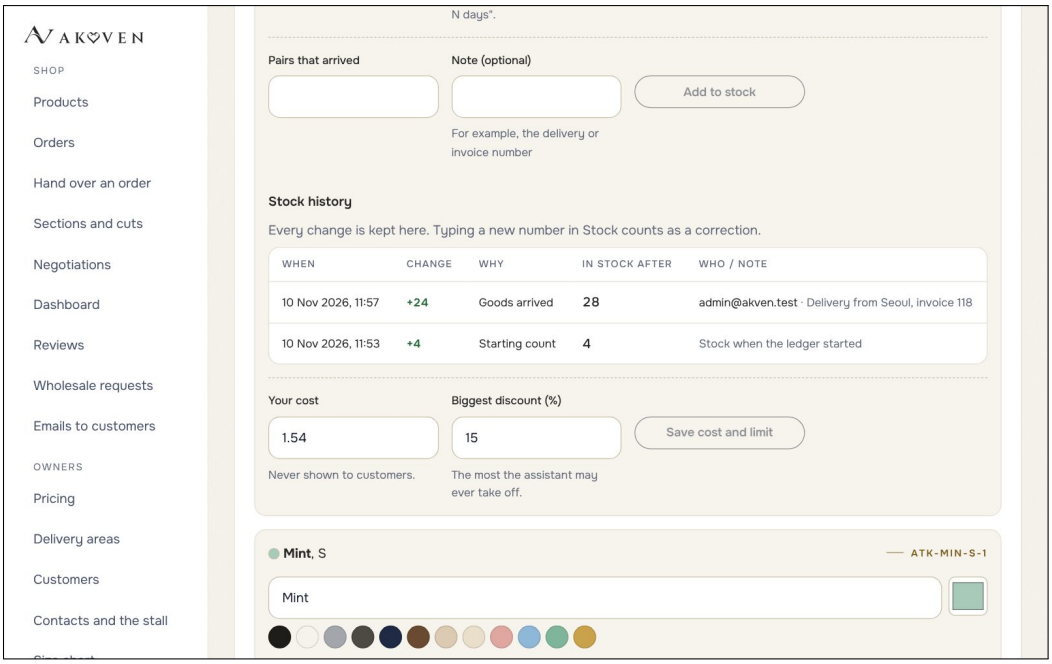


Figure 3.24: Goods arrived, and an option's stock history

Orders and hand-over. *Orders* (Figure 3.25) lists the orders by status; a gift order says how to pack it, a delivery order the area and the fee. An order can be marked as handed over or cancelled (with a refund). *Hand over an order* (Figure 3.26) is the stall's screen: the customer's code, typed or scanned from their QR code, and the last four digits of their phone number. Reserved orders wait in the same list with the amount to take; for them, the team takes the money first, and confirming the handover records the payment and the sale. A reserved courier order is closed from its page with *Paid and handed over*.

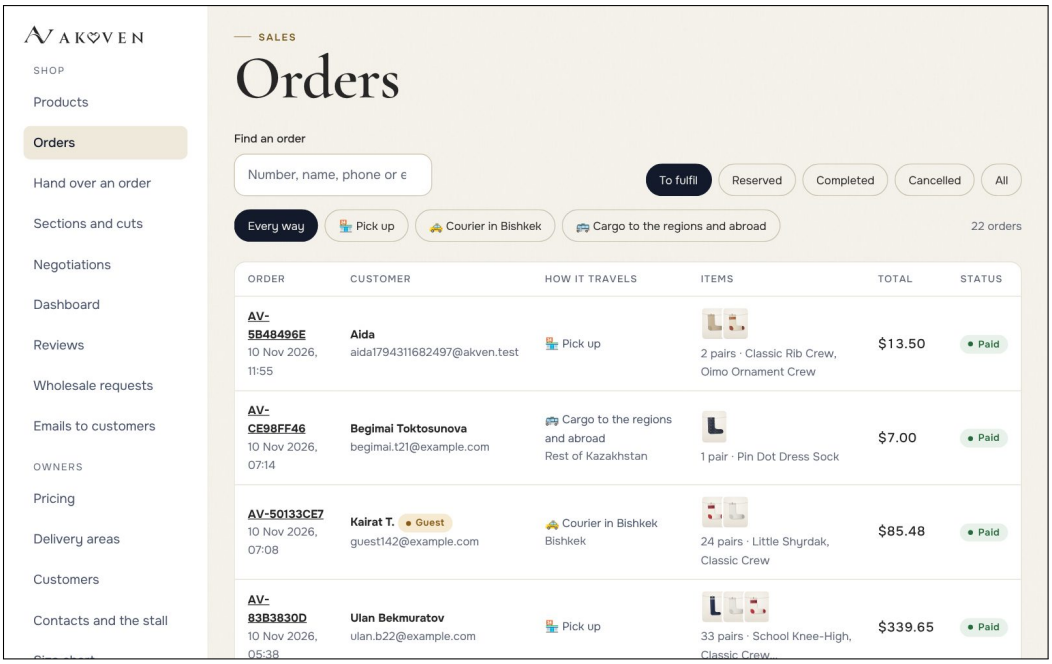


Figure 3.25: Orders

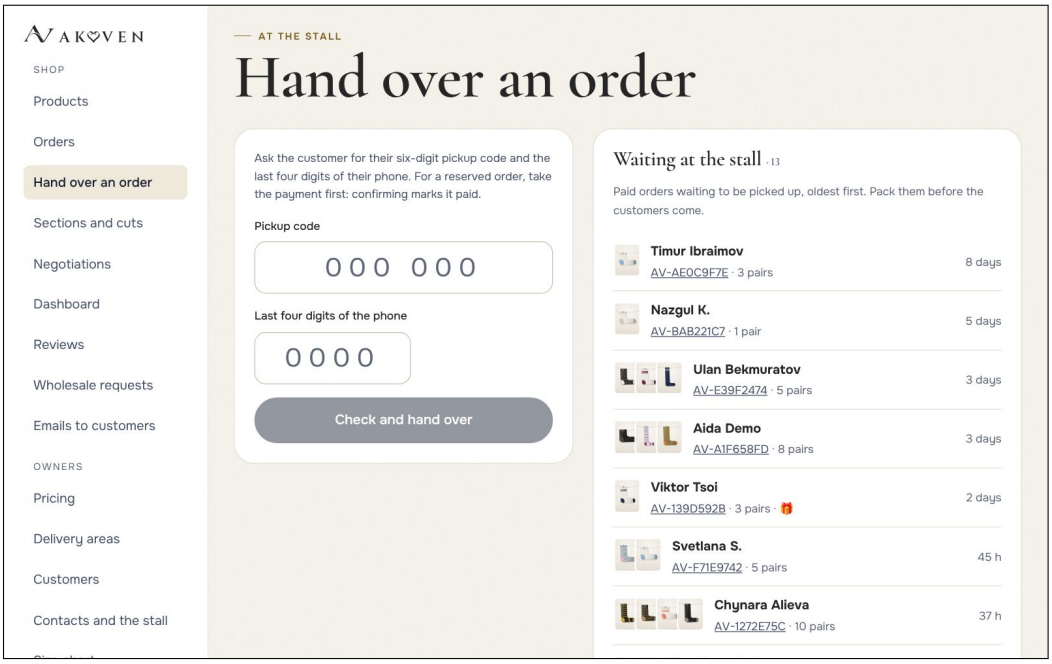


Figure 3.26: Handing an order over at the stall

Sections and cuts are the owners' own groups of the catalogue; they can be added, renamed, hidden, reordered, and deleted when no product uses them.

Negotiations and the dashboard. *Negotiations* (Figure 3.27) lists every conversation with what the assistant proposed and what was allowed. The dashboard (Figure 3.28) shows paid orders, revenue and pairs sold per day, orders waiting at the

stall, socks running out with what is on the way (the owners also get an email the moment a sale leaves a sock at the “few left” level), customers waiting for a sold-out option, and for the assistant the offers per day, how often the limit stepped in and how many offers became orders.

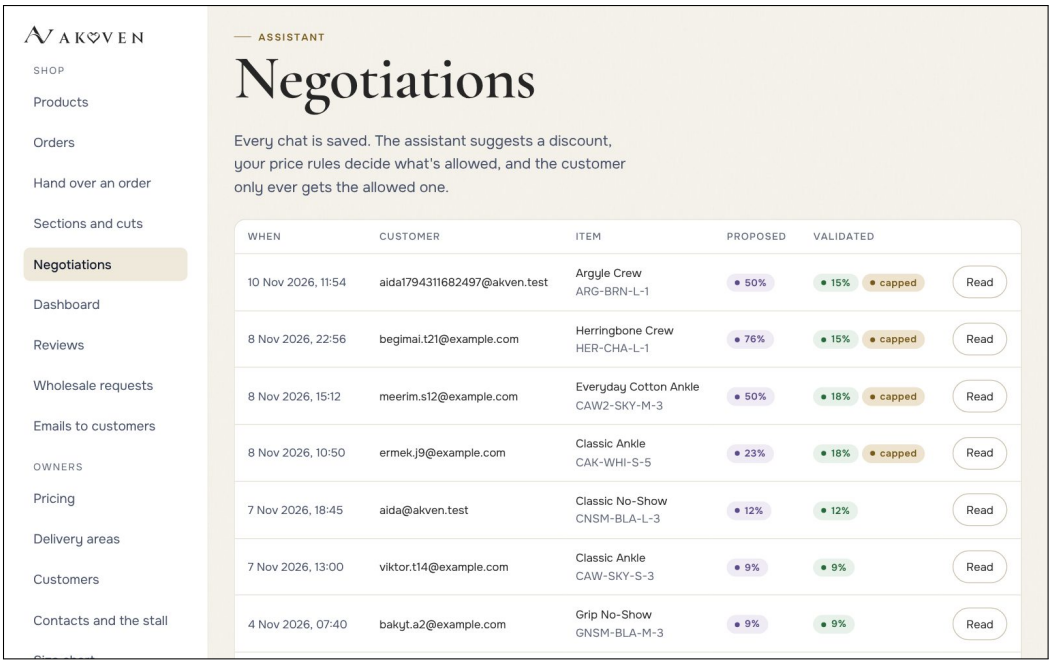


Figure 3.27: Negotiations: proposed and allowed, side by side

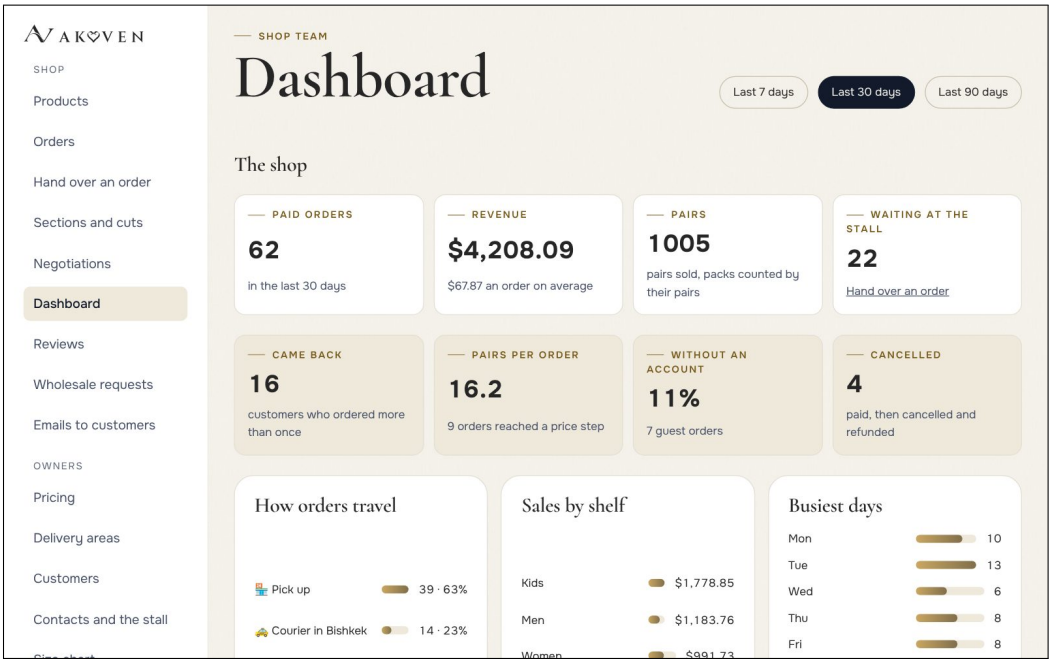


Figure 3.28: The dashboard

Reviews and wholesale requests. Reviews wait for approval before they are shown. *Wholesale requests* (Figure 3.29) is the inbox of requests from shops and companies, newest first: the reference, the company, the city, the pairs and the mix, the contact with a *Reply* button that opens the customer's own WhatsApp, Telegram, phone or e-mail, a note for the family, and the status (new, in progress, answered, closed).

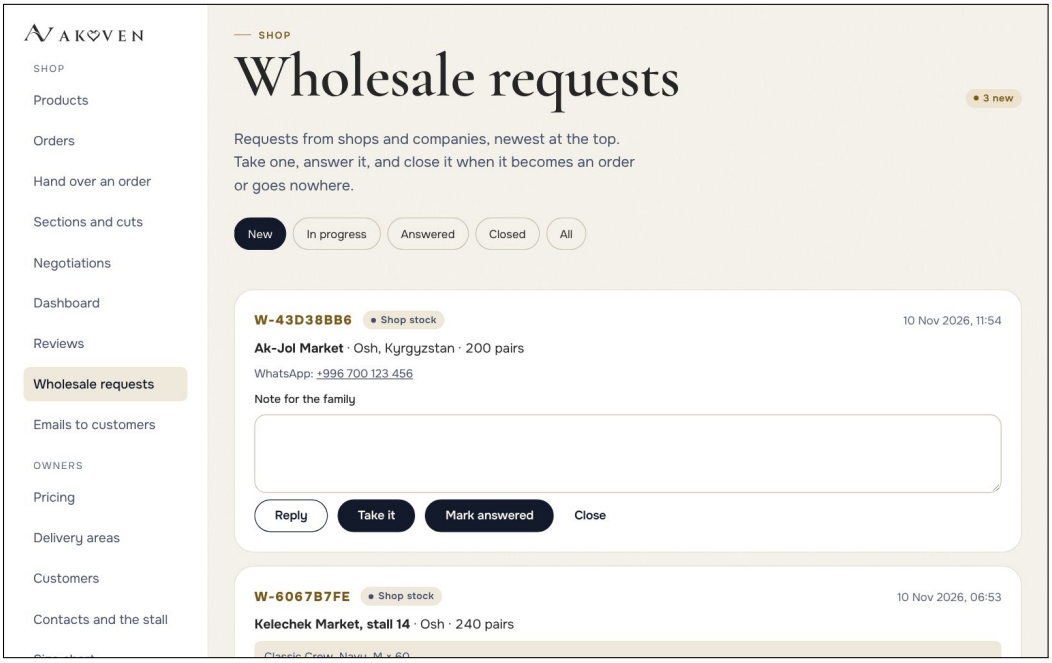


Figure 3.29: The inbox of wholesale requests

3.4.2 For the owners

Pricing (Figure 3.30) sets the minimum order, the ladder of discounts by pairs, and the som and rouble rates with their rounding. The limit of each sock (how much it may be discounted at most) and its cost price are set on the product's options and are visible only to the owners.

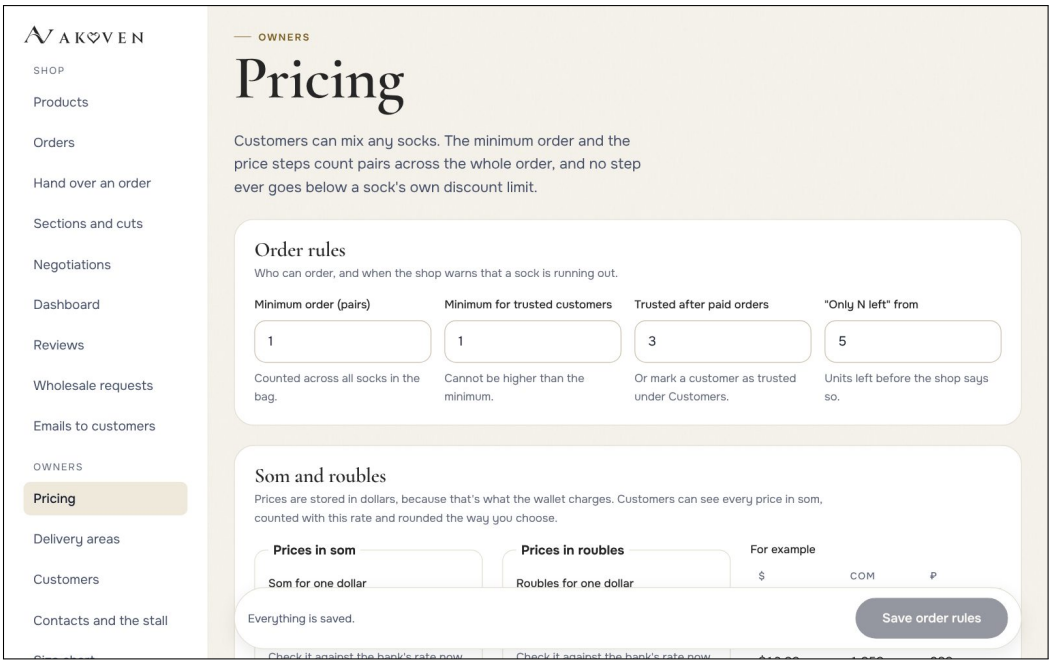


Figure 3.30: Pricing

Delivery areas (Figure 3.31) are where the shop delivers: for each area the country, a name, whether it is a courier area or a cargo carrier, the fee in US dollars, the amount from which delivery is free, the shortest and longest time in days, a note for customers, and for a carrier whether the customer pays it on receipt. An area can be hidden from checkout or deleted; orders that used it keep their copy.

Emails to customers lists every email the shop wrote: to whom, about what, in which language, whether it went out, and the email itself; the order's page shows its own. Without a mail server the page says that the emails are kept and not sent, and *Send again* delivers one once a server is set up.

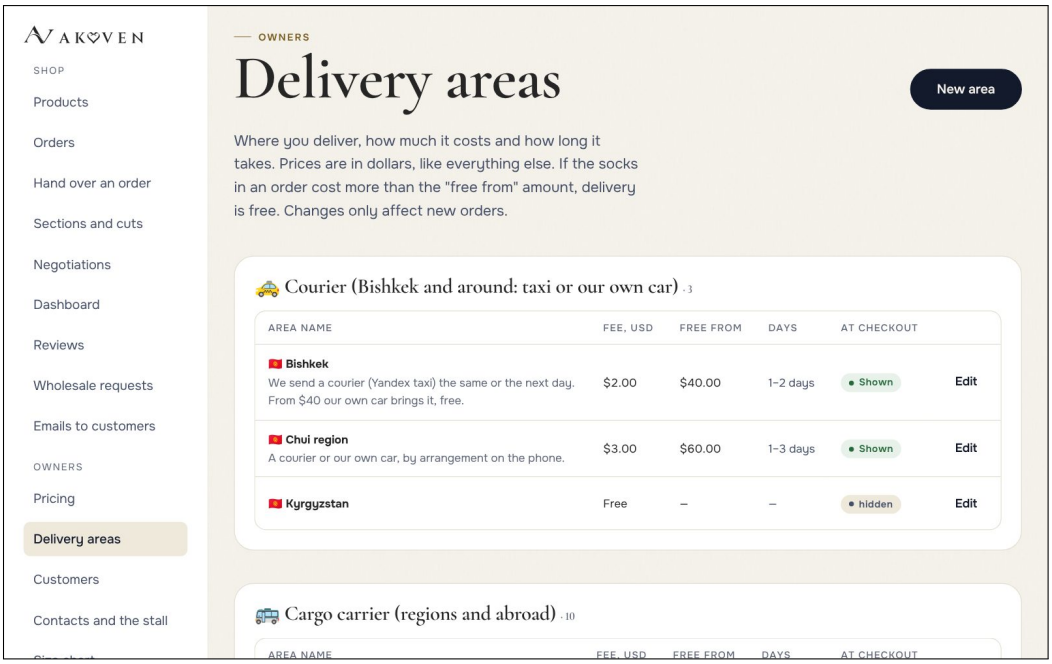


Figure 3.31: Delivery areas

Customers, contacts and the stall, and the size chart. The owners can mark trusted customers, give a customer a new password or switch a fake account off; enter the contacts shown in the shop and the stall's address and opening hours; and edit the size chart from which the shop derives every sock size.

3.4.3 For the super admin

Accounts adds staff and owner accounts, changes their role, disables them or sets a new password; a disabled account or a new password ends its sessions at once. *AI assistant* (Figure 3.32) chooses who answers the chat (the language model or the rules), shows whether a key is set (never the key itself), offers a box to try a message, and repeats the evaluation of Section 5.6 against the live model. The *Audit log* (Figure 3.33) shows every change with who made it and the values before and after.

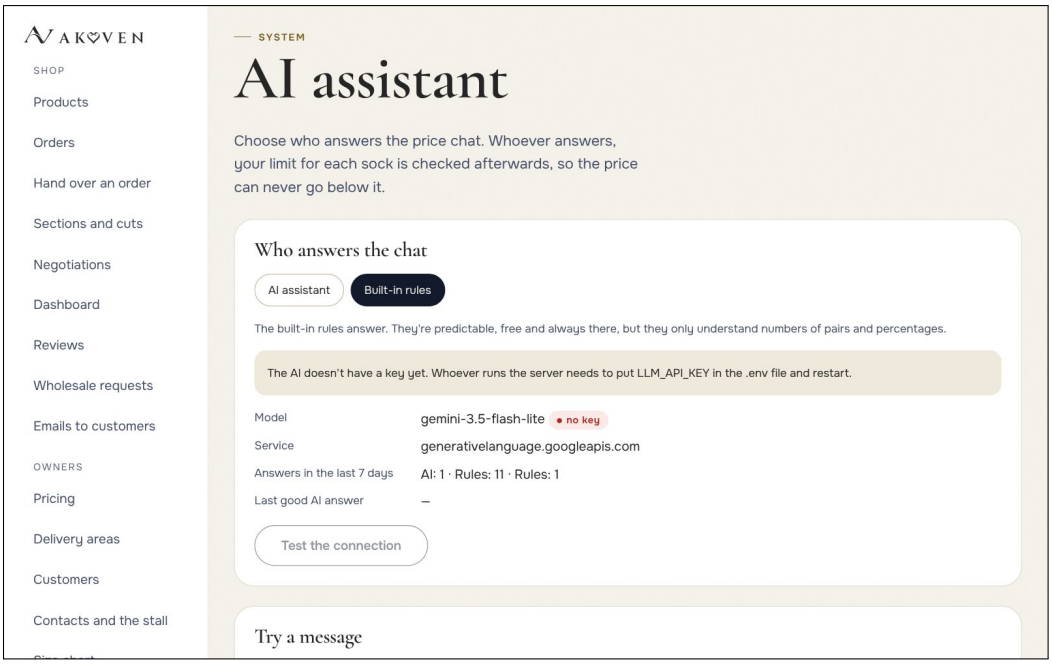


Figure 3.32: Choosing and trying the assistant

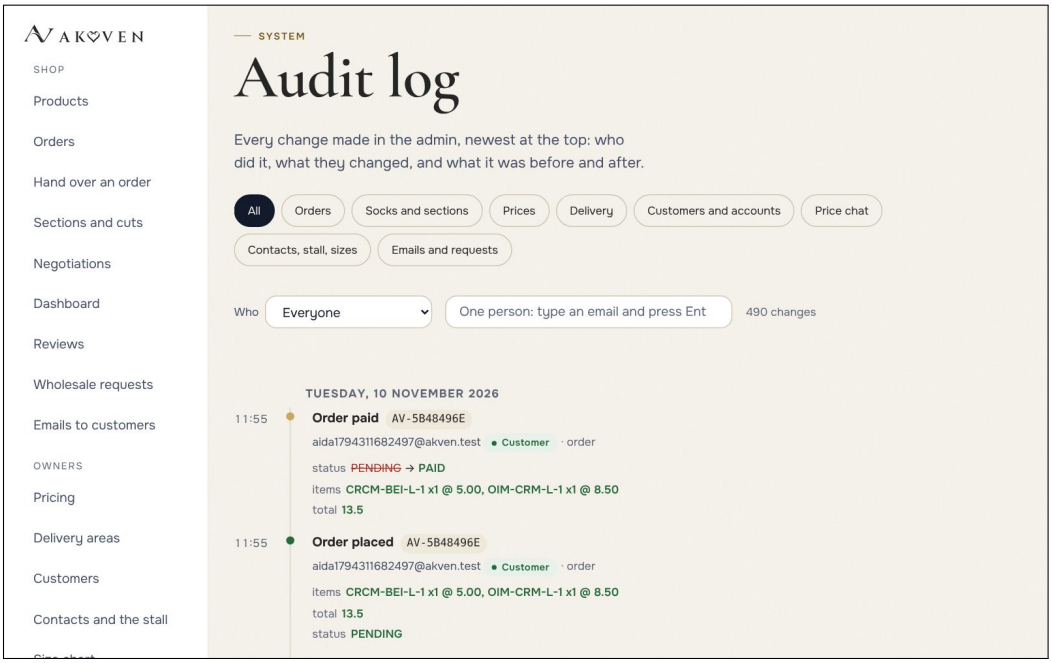


Figure 3.33: The audit log

3.5 Troubleshooting

The page shows an older version. Reload without the cache (Ctrl+Shift+R, or Cmd+Shift+R on a Mac); an installed app updates on its next start.

“Too many attempts” at sign-in. After five wrong passwords for an account from one address, sign-in waits for fifteen minutes; the message says how long.

The chat says to wait a little. A customer can send twenty messages in ten minutes.

The assistant answers but the language model is not used. Without `LLM_API_KEY`, or when the model does not answer in time, the rules answer; the super admin sees which one answered under AI assistant.

A delivery country is missing at checkout. The country has no area shown at checkout; the owners add one under Delivery areas.

The backend does not start. Check that port 8080 and 5432 are free and that Docker has enough memory; the backend log names the reason.

3.6 Limitations

Payment is simulated: no money moves and the wallet does not contact Apple or Google. The shop sends no e-mails or SMS; customers follow their orders in the account. Product pictures are drawings until the family photographs the real socks. The section and colour names come from the owners’ data and are shown as entered, in one language.

Chapter 4

Developer documentation

This chapter describes how AK&VEN is built: the overall design and the reasons for it, the technologies, the backend packages, the data model, the API, security, the pricing and negotiation logic, the remaining domain features, the frontend, and how the system is built and deployed. Testing and evaluation follow in Chapter 5.

4.1 Overview

The system has three layers (Figure 4.1):

1. **Client:** one progressive web application in React and TypeScript. The shop and the administration are the same application with role-gated routes, not two programs.

2. **API:** one Spring Boot service that holds all business rules: catalogue, pricing, negotiation, orders, payment through a wallet interface, accounts and the audit log. The browser never decides a price.

3. **Data:** PostgreSQL 16 with the pgvector extension. The schema is created and changed only by versioned Flyway migrations [16]; Hibernate validates it at start and never alters it.

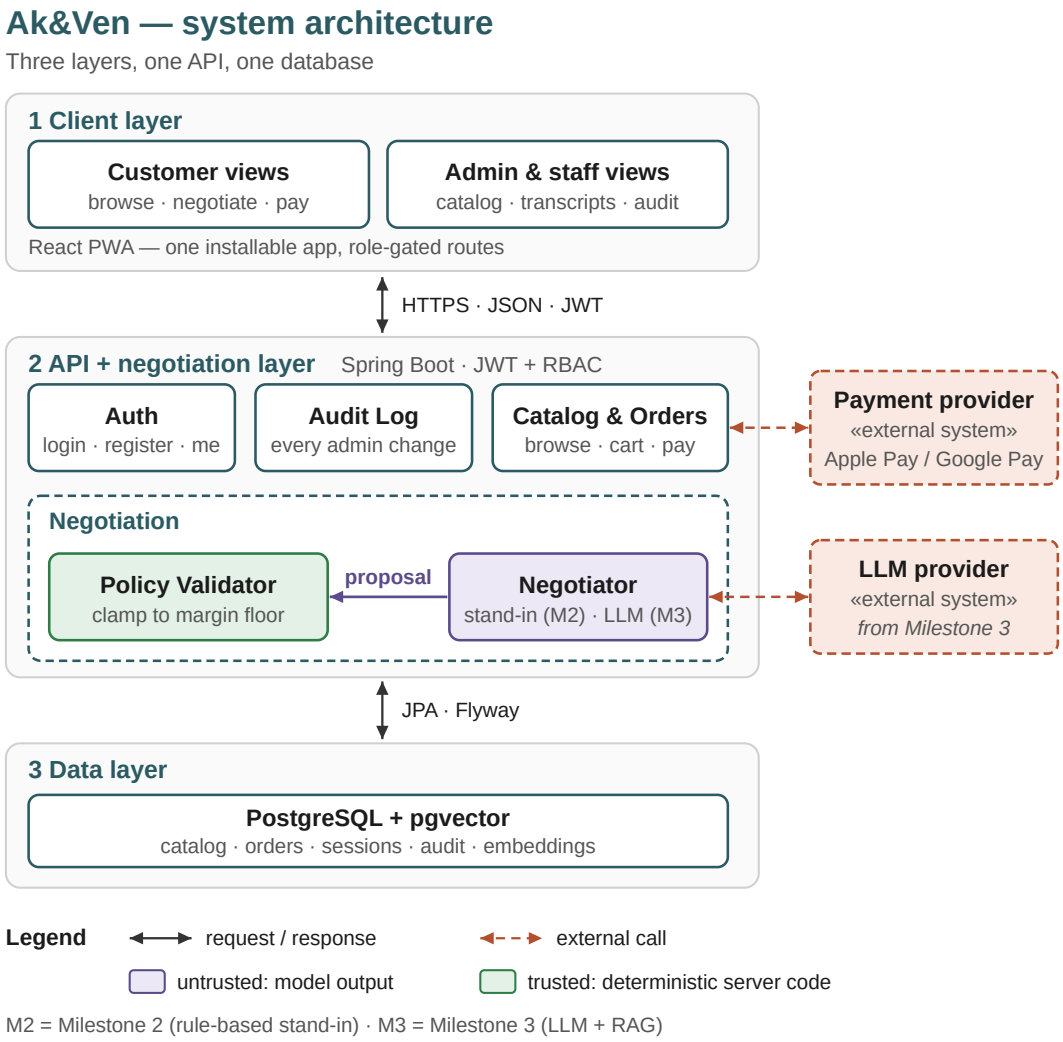


Figure 4.1: Architecture overview

The shape is deliberately modest. Open-source platforms such as Shopizer, Medusa and Saleor (Section 1.2) solve many shops on many servers; this project serves one family stall with a few thousand options, so one service and one database are enough, and every additional moving part (a message broker, a separate vector store, microservices) would add operational cost without benefit. What the platforms contributed is the idea that the administration is only another client of the same API with more rights.

4.2 Technology stack

Part	Technology and reason
Backend	Java 17, Spring Boot 3.5 [17], Spring Security [18], Spring Data JPA with Hibernate 6, Bean Validation. A mature, strongly typed stack with declarative transactions and method security.
Database	PostgreSQL 16 [19] with pgvector; Flyway migrations. Check constraints, generated columns and partial indexes let the database repeat the business rules.
API style	REST with JSON; errors as RFC 7807 problem details [20]; JWT bearer tokens [21]; OpenAPI documentation generated at `/swagger-ui`.
Frontend	React 18 [22], TypeScript [23], Vite [24], React Router; no UI framework: the design system is written in CSS for the brand. A service worker and a web app manifest make it a PWA.
Assistant	A `Negotiator` interface with a rule-based implementation and a language-model implementation over any OpenAI-compatible chat API (Gemini by default [14]).
Tests	JUnit 5 [25], MockMvc, JaCoCo [26]; Vitest [27] and Testing Library; Playwright [28] with axe-core [29] for the browser journey.
Delivery	Docker Compose [15] (database, backend, nginx web server); CI on GitHub Actions and on the ELTE GitLab.

Table 4.1: Technology stack

4.3 Repository orientation

```
backend/            Spring Boot application
  src/main/java/com/akven/thesis/
     audit auth catalog common config delivery family media
        negotiation
     order payment pricing review saved settings shop sizes stock
        user wholesale
  src/main/resources/db/migration   schema migrations (always
     applied)
  src/main/resources/db/seed        demo data (demo profile only)
frontend/           the shop and the administration (React +
   TypeScript)
  src/api  auth  cart  brand  components  pages  admin  shop  i18n
      styles  test
  scripts/          product pictures, logo, icons, screenshots, end
     -to-end test
docs/               requirements, architecture, diagrams, this
   thesis
```

```
docker-compose.yml   .github/workflows/ci.yml   .gitlab-ci.yml
```

Code 4.1: Repository layout

The backend has about 10,800 lines of Java in 21 packages, organised by business feature rather than by technical layer: each package holds its entity, repository, service, controller and data-transfer records. The frontend has about 17,200 lines of TypeScript and CSS. Tests add about 5,300 lines on the backend and 1,500 on the frontend, plus the browser journey.

4.4 System design

4.4.1 Layers inside the backend

Each request passes through the same layers:

Filters. A correlation-id filter gives every request an id that appears in the logs and in audit entries; the JWT filter authenticates the caller from the token and reads the account from the database on every request.

Controllers. Thin REST controllers that validate the request body (Bean Validation) and pass the caller's email, taken from the token and never from the body, to a service.

Services. All business rules, in transactions; owner-only and staff rules are enforced here with `@PreAuthorize`, so they hold for every caller and not only for one URL.

Repositories and entities. Spring Data repositories; entities protect their own state transitions (an order cannot skip from pending to handed over).

Errors are one format: a business-rule exception becomes 400, a missing record 404, a conflict or a lost optimistic-lock race 409, a rate limit 429 with `Retry-After`, and a declined payment 402, all as problem details whose text the frontend shows to the user.

4.4.2 Roles

There are four roles: `CUSTOMER`, `STAFF`, `ADMIN` (the owners) and `SUPER_ADMIN`. A Spring role hierarchy makes `SUPER_ADMIN` include `ADMIN`, so owner rules need not

mention the super admin. URL rules give the public catalogue, sign-in, registration, the price quote, the shop's information, delivery areas, wholesale requests, checkout and a guest's own order (which the service opens only with its private link) to everyone, `/api/admin/**` to the team, and everything else to signed-in users; method security refines this inside the services.

4.5 Data model

Figure 4.2 shows the core tables; the full schema of 43 Flyway versions (24 schema migrations and 19 demo-data migrations) is generated from the database into the `docs/diagrams` folder.

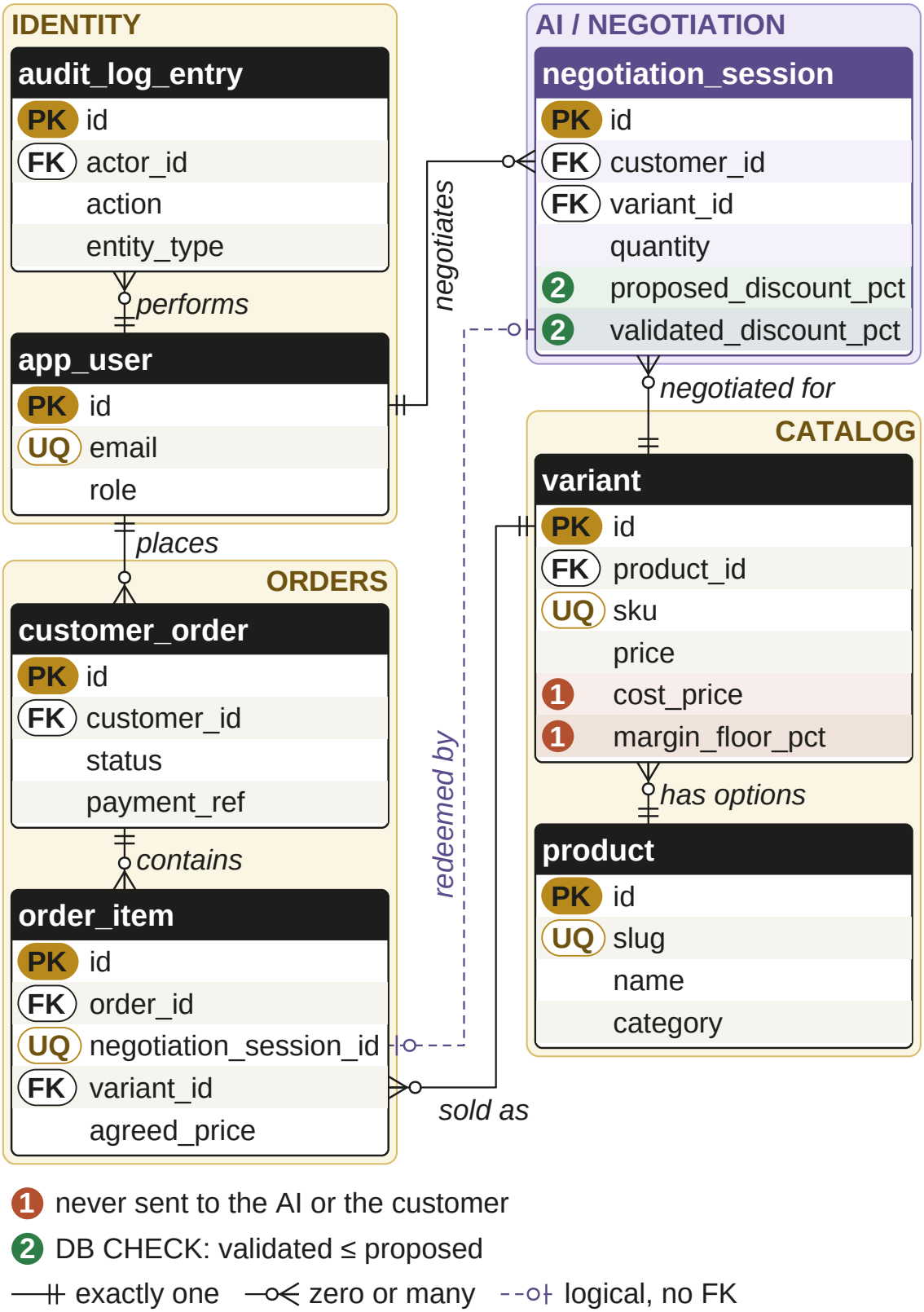


Figure 4b. Core data model: 7 of 10 tables, key columns.

Figure 4.2: The core of the data model

Table	Purpose
product, variant	A sock style and its sellable options (size, colour, pack, price, stock, reserved stock, cost price and limit). Products are removed softly so old orders stay intact.
catalog_term	The owners' sections and cuts.
product_image	Ordered pictures, each optionally tied to one colour.
customer_order, order_item	Orders with their status, payment reference, pickup code, gift packing and delivery copy; lines keep a snapshot of the name, picture and price paid and which rule set it.
negotiation_session, negotiation_turn	A conversation about one option and every message of it, with the proposed and allowed discount and the assistant that answered.
price_tier, shop_policy	The ladder of discounts by pairs, the minimum order, exchange rates.
delivery_zone	Delivery areas: courier or cargo carrier, fee, free-from amount, days, a note, and whether the carrier is paid on receipt.
outbound_email	The outbox: every email to a customer (and the owners' low-stock alerts), its language, state and attempts.
wholesale_request	Requests from shops and companies with their status.
customer_details	Contact and delivery details a customer asked to keep.
family_member, saved_item, product_review, stock_request	Family profiles, saved socks, reviews and waiting for sold-out options.
app_user, app_setting	Accounts with role and state; small run-time settings.
shop_contact, pickup_point, size_chart_row	The shop's contacts, the stall with its hours, and the size chart.
audit_log_entry	Who changed what, with the state before and after as JSON.
product_embedding	Prepared for vector retrieval (pgvector); not used yet.

Table 4.2: The main tables

4.5.1 Integrity in the database

Several rules are enforced by the database as well as by the code, so that a bug in the service layer cannot write an impossible row:

- `available_qty` is a generated column (`stock_qty - reserved_qty`) and `CHECK (stock_qty >= reserved_qty)` keeps stock from going below what is reserved.
- `CHECK (validated_discount_pct <= proposed_discount_pct)` on negotiations: the allowed discount can never exceed the proposal.
- A paid order must have a payment reference; a delivery order must have an address; a negotiated offer can be used by one order line only (a unique index).
- Optimistic locking (a `version` column) on accounts, products, variants and orders turns a lost-update race into an error instead of a silent overwrite [30].
- Delivery fees, review ratings, wholesale quantities, contact kinds and statuses are checked by `CHECK` constraints; e-mail addresses are unique regardless of case.

An order keeps a copy of everything that may change later (the product name and picture, the price paid, the delivery area's name, fee and days), so editing or deleting a product or a delivery area never changes an old receipt.

4.6 REST API

The API has 116 endpoints. Table 4.3 groups them; the full list with request and response shapes is generated at `/swagger-ui/index.html`.

Endpoints	Purpose
Public	
`GET /api/products`, `/facets`	Catalogue with filters and live counts per section, cut, size, colour.
`GET /api/products/{slug}`	A product with options and pictures, never cost or limit.

Endpoints	Purpose
`POST /api/cart/quote`	Prices a bag: lines, collection step, minimum, next step.
`GET /api/pricing`, `/api/sizes`	Public price ladder; size chart and size finder.
`GET /api/shop/info`, `/delivery-zones`	Contacts, the stall, opening hours; delivery areas.
`POST /api/wholesale/requests`	A wholesale or company request (rate-limited).
`POST /api/auth/register`, `/login`	Accounts and tokens (rate-limited).
Signed-in customer	
`POST /api/negotiate`	One message of a negotiation.
`POST /api/orders`, `GET /api/orders`	Checkout (with `Idempotency-Key`); own orders; cancel.
`/api/family`, `/api/saved`, `/api/account/details`	Family, saved socks, kept details.
`/api/reviews`, `/api/stock-requests`	Own review; waiting for sold-out options.
Team (staff and above)	
`/api/admin/products`, `/variants`, `/terms`	Catalogue, options, pictures, sections and cuts.
`/api/admin/orders`, `/handover`	Orders, hand-over with code, fulfil, cancel.
`/api/admin/negotiations`, `/summary`	Transcripts, statistics, the dashboard.
`/api/admin/reviews`, `/wholesale`	Moderation; the wholesale inbox.
Owners	
`/api/admin/pricing`, `/delivery-zones`	Ladder, minimum, rates; delivery areas.
`/api/admin/customers`, `/shop`, `/sizes`	Customer accounts, contacts and stall, size chart.

Endpoints	Purpose
Super admin	
`/api/admin/accounts`, `/assistant`, `/audit`	Team accounts, assistant switch and evaluation, audit log.

Table 4.3: The API by audience

4.7 Security

- **Passwords** are hashed with BCrypt; a password is limited to 72 characters, the length BCrypt reads, so a longer one is refused rather than silently cut.
- **Tokens.** Each installation signs its tokens with its own random 384-bit key unless one is configured; no key is in the repository. The JWT filter reads the account on every request, so a disabled account or a new password ends its sessions at once, and the role comes from the database, not from the token.
- **Brute force and abuse.** Five failed sign-ins per account and address lock that pair for fifteen minutes; sign-ups, chat messages and wholesale requests are rate-limited in sliding windows. The limiters forget old entries once they hold many, so a flood of different addresses cannot grow memory without end.
- **The visitor's address** is taken from `X-Forwarded-For` only when the request comes through a proxy on a private network, so limits count visitors and not the web server.
- **Input.** Every request body is validated; free text (reviews, gift cards, wholesale notes, saved details) is stored as plain text with control characters removed and is rendered by React as text, never as markup. The orders spreadsheet is protected against CSV injection [31]. Uploaded pictures are accepted by their content, not their name, with random stored names; SVG is refused.
- **Secrets and private data.** Cost prices and limits never appear in customer responses (explicit DTOs, asserted by tests). The language model's key is read from the environment and never stored, shown or logged; real phone numbers come from the environment.

- **Headers.** The API sends a strict Content-Security-Policy, `no-referrer` and a permissions policy; the web server sends a CSP that allows only the site's own scripts, `nosniff` and `frame-ancestors 'none'`. CORS allows only configured origins, without credentials.

- **Audit.** Every change by the team is written to the audit log in the same transaction (the audit service requires an existing transaction), with the correlation id of the request.

4.8 Pricing and the PolicyValidator

4.8.1 Collection pricing

A bag is priced as one collection. The owners set a minimum number of pairs and a ladder of steps, for example 3% off every pair from 20 pairs, 5% from 30, 8% from 50 and 10% from 100. Packs count with their number of pairs. The `CollectionPricing` component counts the pairs of the whole bag, finds the step reached and the next one, and the quote tells the customer how many more pairs reach it. The quote is public and is recomputed on every change; the browser only shows it.

4.8.2 The validator

Every discount, whether it comes from the ladder or from a negotiation, passes through one method before it can touch a price (Code 4.2). Each option has a limit (`marginFloorPct`) set by the owners: the largest discount it may ever get. The method is deliberately trivial: it is easy to test exhaustively, it does not depend on the model, and it cannot be talked into anything.

```
1
2 /**
3  * The safety gate described in the architecture doc: the ONLY code
       allowed to
4  * turn a proposed discount into something that can touch an order.
       Deliberately
5  * plain, deterministic, and independent of the LLM call -- treat
     whatever the
6  * negotiation service hands in as untrusted input, the same way
     you'd never
```

```
 * trust a client-submitted price.
 */
@Component
public class PolicyValidator {

    /**
     * @param proposedDiscountPct what the LLM suggested -- never trusted directly.
     * @param variant the SKU being negotiated, whose marginFloorPct is the hard limit.
     * @return the discount actually allowed: min(proposed, marginFloorPct), never negative.
     */
    public BigDecimal clamp(BigDecimal proposedDiscountPct, Variant variant) {
        if (proposedDiscountPct == null || proposedDiscountPct.signum() < 0) {
            return BigDecimal.ZERO;
        }
        return proposedDiscountPct.min(variant.getMarginFloorPct());
    }
}
```

Code 4.2: The PolicyValidator

At checkout, `PricingService` decides the price of each line (Code 4.3): the better of the negotiated offer and the collection step is chosen, never both, and the result passes through the validator again. A negotiated offer is accepted only if it belongs to the same customer and option, has not expired (24 hours), is used for at least the negotiated number of pairs, and has not been used before; the line records which rule set its price and whether the limit stepped in.

```
    private PricedLine decide(Variant variant, BigDecimal tier, BigDecimal negotiated, UUID sessionId) {
        BigDecimal list = variant.getPrice();
        boolean offerWins = negotiated.compareTo(tier) > 0;
        BigDecimal wanted = offerWins ? negotiated : tier;
        BigDecimal pct = policyValidator.clamp(wanted, variant);
        DiscountSource source = pct.signum() == 0 ? DiscountSource.NONE : offerWins ? DiscountSource.NEGOTIATED :
```

```
            DiscountSource.TIER;
7       BigDecimal unit = list.multiply(BigDecimal.valueOf(100).
            subtract(pct)).divide(BigDecimal.valueOf(100), 2,
            RoundingMode.HALF_UP);
8       // The offer stays linked even when the tier was better: it
             was presented at checkout and is used up with this
            order.
9       return new PricedLine(list, pct, unit, sessionId, tier,
            source, pct.compareTo(wanted) < 0);
10  }
```

Code 4.3: Deciding the price of an order line

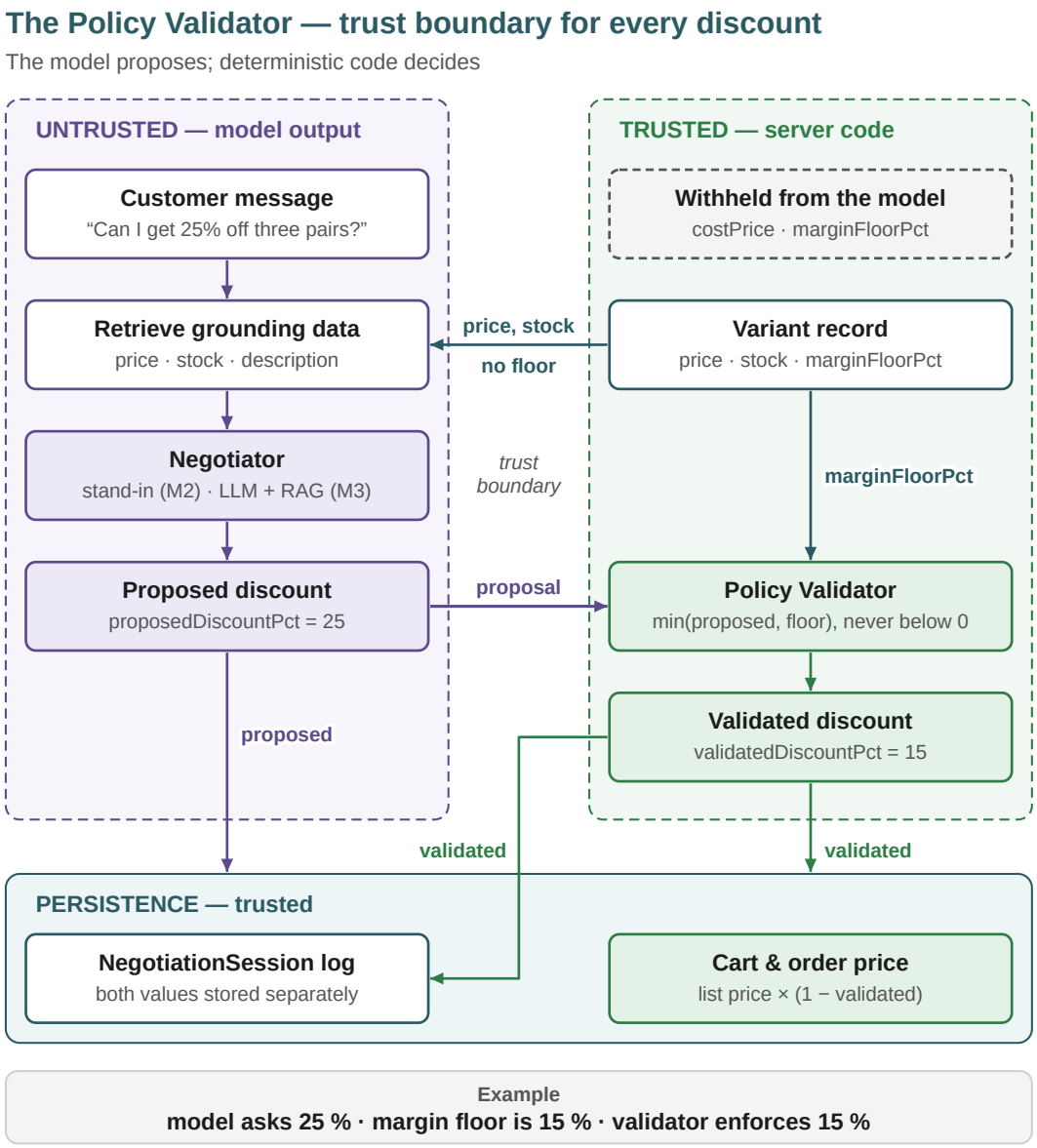


Figure 4.3: The validation path: the proposal is untrusted until the validator has clamped it

4.9 The negotiation assistant

4.9.1 Design

`NegotiationService` handles one message (Figure 4.4). It builds a `NegotiationContext` with the product, its price, the quantity, the public price ladder, the stock status, the conversation so far and the customer’s message; by construction the context has no field for the cost price or the limit, so they cannot reach the model. It asks a `Negotiator` for a proposal (a discount and a reply), clamps

the discount with the validator, stores the turn with both numbers, and fills the reply's placeholders with the allowed numbers.

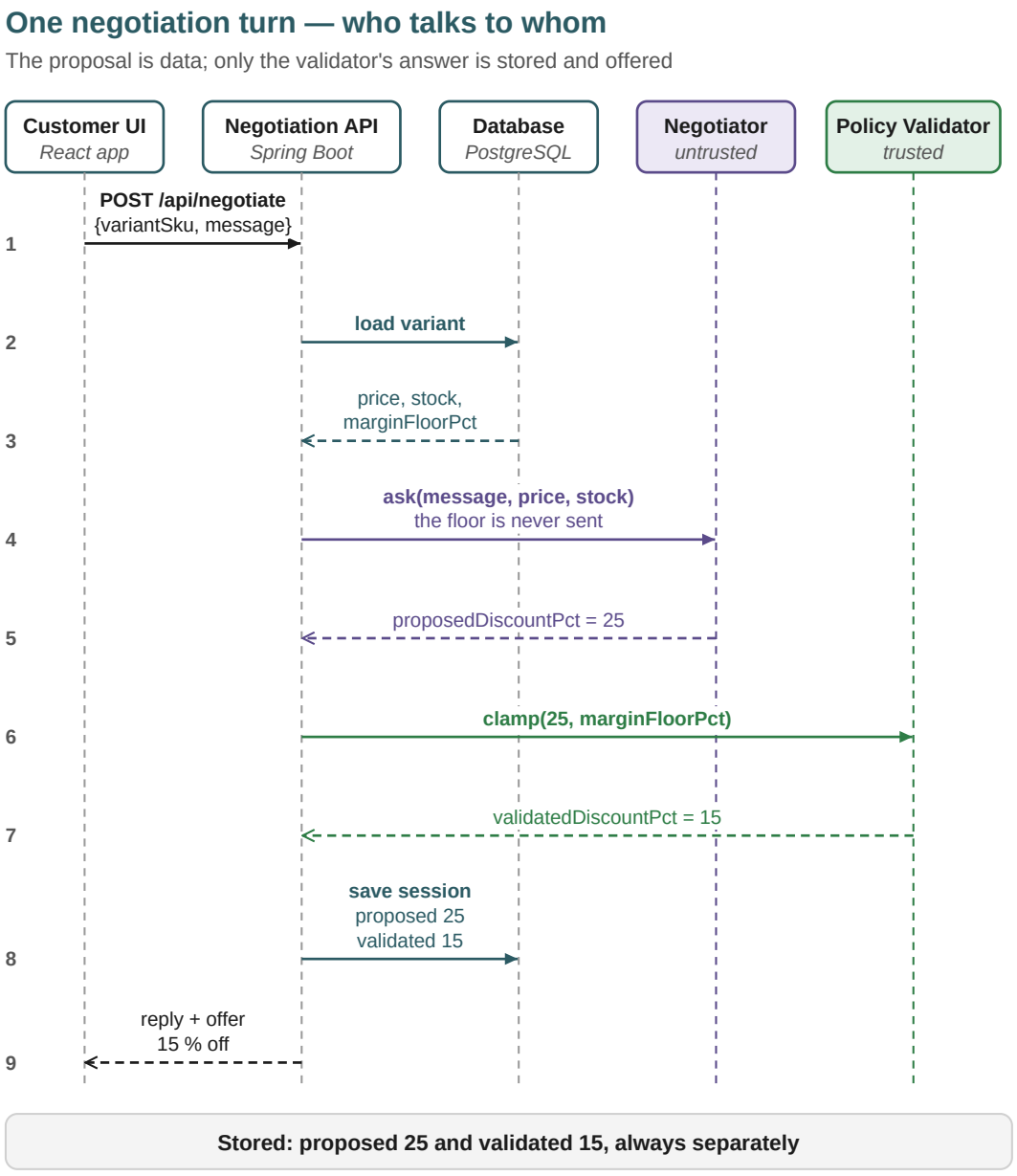


Figure 4.4: One negotiation turn

There are two negotiators behind the same interface:

- `RuleBasedNegotiator` answers instantly from rules about quantity and reasons; it is deliberately willing to over-promise, so that tests show the validator stepping in.
- `LlmNegotiator` calls a language model through `OpenAiCompatibleChatClient`. It is wrapped by `FallbackNegotiator`,

which answers with the rules if the model fails, is slow or returns something unusable; `SwitchingNegotiator` lets the super admin choose at run time.

4.9.2 Defences of the language-model negotiator

1. **No secrets in the prompt.** The model never sees the limit or the cost; it can only propose.
2. **Grounding.** `ProductKnowledge` selects the product facts relevant to the question (fabric, care, origin, pack, size); the prompt tells the model to answer only from them and to say it will ask the family otherwise.
3. **The message is data.** The customer's text is fenced in `<customer_message>` tags and the system prompt says it is data, not instructions; claims to be the owner or a developer are to be ignored.
4. **Structured output.** The model must answer with a JSON object of a discount and a reply; anything else is a failure that falls back to the rules.
5. **No numbers in its words.** The reply may not contain any number, price or percentage in digits or words in English, Russian or Kyrgyz (Code 4.4); it writes `{pct}` and `{price}` and the shop fills in the *allowed* numbers. A reply that states a number of its own is replaced by a template.
6. **Clamp and record.** The discount is clamped and both numbers are stored; the database refuses a turn in which the allowed discount exceeds the proposal.

```
1    /**
2     * A reply is safe when, apart from the shop's placeholders, it
          states no number, price or percentage, in digits or
3     * words, in English, Russian or Kyrgyz. The one exception is
          the customer's own quantity ("6 pairs", "six pairs",
4     * "\u043f\u044f\u0442\u044c \u043f\u0430\u0440", "\u0431\u0435
          \u0448 \u0436\u0443\u043f"), which is just repeating the
          question.
5     */
6    static boolean isSafe(String reply, int quantity, String
         language) {
7        if (reply.isBlank()) return false;
8        String lang = Replies.language(language);
```

```
9         String stripped = reply.replace("{pct}%", " ").replace("{
              pct}", " ").replace("{price}", " ").toLowerCase(Locale.
              ROOT);
10        if (quantity >= 1 && quantity < WORDS_EN.length) stripped =
               stripped.replaceAll("\\b" + WORDS_EN[quantity] + "\\b
              (?=\\s+pairs?\\b)", " ");
11        // The customer's own quantity in digits ("30 pairs", "30 \
              u043f\u0430\u0440", "30 \u0436\u0443\u043f") is only
              repeating the question too.
12        if (quantity >= 1) stripped = Pattern.compile("(?<![\\d.,])
              " + quantity + "(?=\\s+(pairs?\\b|\u043f\u0430\u0440|\
              u0436\u0443\u043f))", Pattern.UNICODE_CASE).matcher(
              stripped).replaceAll(" ");
13        if (quantity >= 2 && quantity < WORDS_RU.length) {
14            stripped = Pattern.compile("(?<!\\p{L})(" + WORDS_RU[
                  quantity] + ")(?=\\s+\u043f\u0430\u0440)", Pattern.
                  UNICODE_CASE).matcher(stripped).replaceAll(" ");
15            stripped = Pattern.compile("(?<!\\p{L})(" + WORDS_KY[
                  quantity] + ")(?=\\s+\u0436\u0443\u043f)", Pattern.
                  UNICODE_CASE).matcher(stripped).replaceAll(" ");
16        }
17        if (NUMBERISH_EN.matcher(stripped).find() || stripped.
              contains("{") || stripped.contains("}")) return false;
18        if ((lang.equals(Replies.RU) || lang.equals(Replies.KY)) &&
               NUMBERISH_RU.matcher(stripped).find()) return false;
19        return !(lang.equals(Replies.KY) && NUMBERISH_KY.matcher(
              stripped).find());
20    }
```

Code 4.4: A reply may state no number of its own (Cyrillic shown as Java Unicode escapes)

4.10 Orders, stock and payment

Checkout (`OrderService`) is the place where races matter: two customers may buy the last pair at the same moment, and a double click must not charge twice. The method therefore:

1. returns the earlier order when the same `Idempotency-Key` is seen again from the same customer (a unique index backs this);

2. locks the variants in the bag with `SELECT ... FOR UPDATE` in SKU order before looking at the stock, so two carts in opposite orders cannot deadlock and the last pair goes to one buyer;

3. prices every line on the server and reserves the stock;

4. adds the delivery fee of the chosen area (zero above its free-from amount) to the total;

5. charges the wallet through the `PaymentProvider` interface with only a token; a decline or an outage cancels the order and releases the stock, and that outcome is committed;

6. marks the order paid with the provider's reference, gives a pickup code, and audits each step.

The order's status machine lives in the entity (Figure 4.5 gives it in words): an order starts *pending* while it is being paid; a wallet payment makes it *paid*, and the team's handover makes it *fulfilled*; a pending, paid or reserved order can be *cancelled* (a paid one with a refund and its pairs back on the shelf). The simulated wallet accepts only token shapes of Apple Pay and Google Pay and refuses anything that looks like a card number.

Reserve now, pay at handover. Most customers of the stall pay in cash or with a bank app, and an online payment provider for Kyrgyzstan is future work. So a customer may also choose to pay in person. The order then becomes *reserved*: its pairs are held exactly as during a wallet payment (`reserved_qty`), so nobody else can buy them, but they are not sold and nothing is charged. A pickup order gets its code at once. At the stall the team checks the code and the last digits of the phone, takes the money and confirms; only then is the order sold, in one transaction: the variants are locked, the pairs leave the shelf, the ledger records the sale, and the low-stock check runs. A courier order is confirmed the same way from its page. A reservation holds its pairs for 48 hours; a scheduled job ends the ones nobody collected, locks each order again first (so a handover at the same moment always wins), gives the pairs back and writes to the customer. Because holding pairs costs the customer nothing, one reservation may hold at most 60 pairs; larger orders go through the wholesale request. The database keeps the rule that a reserved order

always has its deadline (a check constraint), and pickup codes are unique among paid and reserved orders (a partial unique index).

The stock ledger. Stock used to be a single number per variant that each change overwrote, so "why is it seven?" had no answer in the data. Now every change to a variant's stock writes one row in `stock_movement` with its reason (opening count, goods arrived, sale, cancelled sale, count corrected by hand), the order or the person behind it, and the stock right after it. The row is written by the same transaction as the change (`Propagation.MANDATORY` makes a call outside a transaction fail), and rows are only ever added. Two properties follow and are tested: a variant's rows add up to its stock, and each row's *stock after* is the previous one plus its own change, so the history reads like a bank statement. Typing a new number into the stock field is kept as a correction by the difference, which is safe because an edit must name the version it was made on: a sale in between makes the edit fail instead of being overwritten. The owners see the history on each variant, with every sale linked to its order, and record a delivery with "Goods arrived", which also takes the pairs off "on the way".

From	Event	To
(new)	checkout with a wallet	pending
pending	wallet approves	paid
pending	wallet declines or is unreachable	cancelled
(new)	checkout, pay at handover	reserved
paid	handed over	fulfilled
reserved	paid in person and handed over	fulfilled
reserved	time is up, or cancelled	cancelled (pairs back)
paid	cancelled	cancelled (refund, pairs back)

Figure 4.5: The states of an order and what moves it between them

4.11 Further domain features

Delivery areas. `DeliveryService` answers which area an order travels in: the chosen active area, or the first active area of the country. Owners edit areas with checks that the longest time is not shorter than the shortest and that a free area has no free-from amount; every change is audited. A fresh installation starts with one free area per country, so delivery works as before until the owners set prices.

Courier and cargo. An area is a *courier* area (Bishkek and the Chui region, where the family calls a taxi or brings a large order in its own car, free above the area's amount) or a *cargo carrier* (the bus cargo to Osh, carriers to Kazakhstan, Uzbekistan and Russia), which can be paid by the customer on receipt. The order keeps its own copy of the kind and the payment on receipt, like the fee, so the receipt never changes.

Automatic low-stock alerts. When a paid order takes an option from above the owners' "few left" level to at or below it, `OrderService` collects it and `Notifier.lowStock` writes one email per order to every active owner and super admin: which socks, how many are left and how many are on the way, with a link to the dashboard. Only the crossing counts, so later sales of the same option do not repeat the alert, and like every email it is written in the order's transaction. The Telegram bot (future work) can send the same news from the same moment.

Emails through an outbox. `Notifier` writes each email to a customer (confirmed, handed over, cancelled; a wholesale request received) into `outbound_email` *inside the transaction that changes the order*, in the language the customer used at checkout. A change that rolls back therefore sends nothing, and a mail server that is down loses nothing: `EmailSender` delivers the queue every few seconds through any SMTP server, tries a refusal again after 1, 2, 4 and 8 minutes and then marks it failed. Without a mail server nothing leaves the shop and each email is marked *not sent* but stays readable in the admin. Emails hold names and addresses, so they are deleted after 90 days. The texts live in three property files, and a test fails when a sentence is missing in one language. The acknowledgement of a wholesale request carries only its reference and number of pairs, never text the visitor typed, so the public form cannot be used to send someone's words to a stranger.

Checkout without an account. An order has either an account or a guest's email (a database check keeps exactly one). A guest opens the order at `/orders/<id>#t=<key>`. The key is an HMAC-SHA256 of the order id under a key derived from the installation's secret, so nothing is stored, a retried checkout returns the same link, and the key can never pass as a login token. It stands after the `#`, which browsers never send, so it reaches no access log; the page sends it in an

`X-Order-Token` header and the server compares it in constant time. A guest who lost the link gives the email and the order number, and the link is emailed to the order's own address; the answer is the same whether anything matched, so the form reveals no customer. Guest checkouts and link requests are limited per address. Negotiated prices still need an account, because an offer belongs to the account that negotiated it; and guest orders are never joined to an account by email, because an email address is not proof of identity.

Wholesale requests. `WholesaleService` accepts requests without an account. The contact is checked against its kind (a phone number for WhatsApp and calls, a Telegram name or number, an e-mail address); text is cleaned; a hidden trap field, which people never see, makes a bot's request look accepted while nothing is kept; and a sliding window allows five requests per address and sixty for the whole shop per hour. The team moves a request through new, in progress, answered and closed; a picked-up request cannot become new again; every change is audited.

Saved details, family and reviews. Each customer reads and writes only their own family, saved socks and kept details: the services take the account from the token, never an id from the request. Reviews are accepted only from customers whose order with that sock was handed over, are shown only after approval and never show an e-mail address.

Catalogue facets. The filter counts are computed with criteria queries under the same specification as the list, each dimension ignoring its own selection, so the alternatives keep their counts. Sizes and colours count only options that are for sale (and in stock, or of the chosen colour or size, when those are chosen), so "M" means a product that can be bought in M.

4.12 Frontend

4.12.1 Structure

The application starts in `main.tsx`; `App.tsx` declares the routes. Rarely used screens (the administration, checkout, orders, the style guide) are loaded on demand, which keeps the shop's first script small.

`api/` a typed client that adds the token, turns problem details into readable errors with field messages, and one function per endpoint;

`auth/`, `cart/` the session (kept in `sessionStorage`) and the bag (in `localStorage`, validated field by field when read, shared across tabs);

`pages/`, `admin/` the shop's and the administration's screens;

`shop/` features shared by several pages: the family, saved and recently viewed socks, the fitting room, the home page's sections, the wholesale case builder;

`brand/` the logo and the sock drawing, shared with the script that draws the product pictures;

`i18n/` all texts in English, Russian and Kyrgyz (about 1,900 keys per language) with typed keys, so a missing translation is a compile error, and plural rules through `Intl.PluralRules`;

`styles/` design tokens for the light and dark themes and the components.

4.12.2 Design system

The visual language comes from the stall: kraft price tags on strings, the family's handwriting (Caveat) beside a Garamond serif (Cormorant Garamond) and a clear sans-serif (Onest), navy and gold from the logo. Instead of plain boxes, the interface uses objects of the stall: socks pegged on a washing line (a `Clothesline` component that hangs drawn socks by the cuff and lets them swing from the peg), shelves on knitted, ribbed, spotted and wrapping-paper fabrics, contacts as paper tags, the promises in the footer as sewn-in labels, forms as sheets of paper pegged up, and the stall's door with an open or closed sign in Bishkek time. All product pictures are drawn by a script from the same sock shapes the interface uses, hanging on the line, on a transparent ground, so each stands on a calm backdrop chosen by the shop and the shop looks consistent until real photographs (cut out, they would stand on the same backdrops) replace them.

Two countries in one design. The socks are knitted in Korea and chosen in Kyrgyzstan, and the design says so with a small, consistent set of motifs used across the whole site rather than as decoration in one corner. From Kyrgyzstan: shyrdak

felt (the dark bands, the footer, the wholesale and *Find us* headers), the ram's horn (*kochkor müyüz*) as a rosette pressed into the felt and as the heart between AK and VEN in the logo, and the oimo stitch along the edges. From Korea: hanji paper under cards and notes, bojagi patchwork in the hero's tissue paper, and a red dojang seal, which stamps the bestseller ranks and carries each reviewer's initial. Cards that hold the shop's own words are *sewn*: a felt strip on top, a dashed stitch inside the edge. One rule keeps it calm: patterns stay in frames, bands and headers, while the product pictures sit on plain single tones.

A calm shopping path. A design review late in the project found the opposite problem: the shopping pages were as decorated as the story pages, and the stylesheets had grown in layers (92 different font sizes, 42 corner radii, 100 shadows, 27 animations that never stopped). The rule since then is that the stall's warmth lives in a few signature places (the hero, the story, the container scene, the footer) and the path from shelf to checkout stays quiet: every product picture stands on the same paper colour and carries no logo; small labels are one style (small gold capitals after a hairline) and handwriting is kept for the family's own words; on a phone the filters of a shelf fold behind one button, so the first socks show on the first screen. Motion uses three durations (150, 250 and 400 ms): fast for feedback such as a button press or the bag's count, the middle one for panels and pictures, the slow one only for larger changes. Animations that looped without meaning were removed; what still loops tells something (a spinner, the loading thread, the "open now" dot, the lights of the container scene while it is on screen). The home page went from twelve sections to nine; the parts that left it kept their own pages.

Performance. Motion follows two rules that keep it smooth on cheap phones: only `transform` and `opacity` are animated, and nothing that changes the layout runs while scrolling. Measurements in Chromium's performance panel with the CPU slowed four times showed three further costs, each removed:

- the felt and paper textures were SVG noise filters (`feTurbulence`) recomputed on every repaint; they are now small PNG tiles drawn once;
- every product picture carried an SVG drop-shadow filter; the generator now draws the shadow as three offset copies of the outline, so no picture contains a filter;

- endless animations (the swinging socks, the turning stamp) kept the compositor busy off screen; an `IntersectionObserver` pauses them while they are out of view.

Hint layers (`will-change`) on every card were tried and measured worse (more layer work than they saved), so they are used only on the few elements that sway. Every animation stops for visitors who prefer reduced motion. A small `ScrollManager` restores the scroll position on Back and Forward, opens a new page at the top at once, and keeps the position when only a filter in the address changes. Colour contrast is checked by a test against WCAG 2.2 AA [32] for every pair of text and background tokens in both themes.

4.12.3 State and data flow

Server data is loaded with a small `useAsync` hook that cancels requests that are no longer needed; shared client state (the bag, the session, saved socks, recently viewed, the family) lives in small stores read with `useSyncExternalStore`, so several components stay in step without a state library. Filters live in the address bar, which makes every view shareable and the Back button meaningful.

4.13 Running and building

With Docker. `docker compose up -build` starts three services: `db` (PostgreSQL 16 with pgvector), `backend` (the Spring Boot jar, demo profile) and `web` (nginx serving the built frontend and passing `/api` to the backend), with volumes for the data and uploaded pictures.

Without Docker. Start PostgreSQL with the vector extension, then the backend with `mvn spring-boot:run` (profile `demo` for the catalogue) and the frontend with `npm run dev`; Vite passes `/api` to the backend. `npm run art` draws the product pictures again.

Continuous integration. Every push runs six jobs on GitHub Actions, and the same checks on the ELTE GitLab: the backend tests with the coverage gate, the tests on real PostgreSQL, a smoke test of the jar against PostgreSQL, the frontend

type check, tests and build, the browser journey against the real stack, and a Docker Compose smoke test that starts the whole system and checks the security headers.

Live deployment. The same Compose file runs the public demo on a small ARM virtual machine (two cores, 12 GB of memory, Ubuntu 24.04) in a cloud provider's Frankfurt region. The backend and the web server listen only on the machine itself; Caddy in front of them obtains and renews the HTTPS certificate automatically and is the only service open to the internet. The secrets (database password, super-admin account, mail settings and, when it is used, the language-model key) live in an `.env` file on the server that never enters the repository.

4.14 Design trade-offs and known limitations

- **Payment inside the checkout transaction.** With the simulated wallet the charge is instant; a real provider would move outside the transaction (reserve, pay, confirm, with a timeout releasing the reservation).
- **Rate limits in memory.** Right for one backend; several would share them through the database or Redis.
- **Keyword retrieval.** The assistant's facts are selected by keywords; the vector table is prepared for embedding-based retrieval.
- **Offline.** The bag works offline; product pages are not cached, and orders need a connection.

Chapter 5

Testing and evaluation

5.1 Testing strategy

The tests are arranged by what they can prove:

Unit tests prove small rules exactly: the validator, the collection ladder, the limiters, the contact checks, the reply filter of the assistant.

Integration tests run the whole Spring application against an in-memory database through the real security filters (MockMvc) and prove each feature end to end at the API: who may do what, what is stored, what is audited.

Database tests run on a real PostgreSQL 16, because some properties exist only there: migrations, check constraints, generated columns, row locks and the 401/403 behaviour of a real servlet container.

Frontend tests render components with a fake server and check what the user sees and what is sent.

The browser journey starts the real backend and frontend and walks through the owner's, the customer's and the super admin's work in a real browser, with accessibility rules checked on every page.

Table 5.1 summarises the suites as they stand at the end of the project.

Suite	Command	Result
Backend	`mvn verify`	284 tests (11 on real PostgreSQL), 96% line coverage; the build fails below 80%
Real PostgreSQL	`-Dtest=PostgresIntegrationTest`	11 tests over real HTTP on PostgreSQL 16
Frontend	`npm test`	213 tests, type check and production build
Browser journey	`npm run e2e`	29 steps in three languages, axe rules on every page

Table 5.1: The test suites

All four run in continuous integration on every push; a commit is moved to the development branch only when all of them pass.

5.2 Backend tests

The backend has 52 test classes. The most important ones, grouped by the property they show:

Test	What it shows
`PolicyValidatorTest`	No discount above the limit, negative or missing proposals become zero.
`UntrustedAssistantTest`	An assistant that offers 80% is capped at the limit in the chat and at checkout.
`LlmNegotiatorTest`	The prompt holds no secrets; replies with numbers in any of the three languages are replaced; malformed JSON falls back to the rules.
`FallbackNegotiatorTest`, `SwitchingNegotiatorTest`	A failing or slow model is replaced by the rules; the super admin's switch takes effect at once.
`ConversationTest`	A negotiation is a conversation; an offer once made stands; offers expire after 24 hours and apply only to the negotiated number of pairs.

Test	What it shows
`OrderControllerTest`	Server-side prices, a tampered offer re-clamped, idempotent checkout, declined and failed payments that release stock, cancellation with refund and restock.
`OrderConcurrencyTest`	Many threads buying the last pair: exactly one succeeds; opposite carts do not deadlock.
`CollectionPricingTest`, `SomPricesTest`	The ladder, the minimum and the exchange rates.
`DeliveryZoneTest`	Owners set areas; checkout charges the area's fee or none above its free-from amount; a fee sent by the browser is ignored; old receipts keep their area after it changes or is deleted.
`WholesaleRequestTest`, `WholesaleRateLimiterTest`	Requests without an account, checked contacts, the bot trap, per-address and shop-wide limits, statuses and audit.
`CustomerDetailsTest`	A customer keeps, sees and forgets only their own details.
`CatalogControllerTest`	Filters, facet counts per dimension, sizes and colours counted for what can be bought, cost and limit never in public responses.
`AdminCatalogControllerTest`	Staff may edit prices and stock but not limits; every change is audited.
`StockLedgerTest`	Every way the stock can change leaves one row with its reason; a variant's rows add up to its stock and each row's "stock after" follows from the one before.
`PayAtHandoverTest`	A reservation holds its pairs without selling them; they are sold only when paid at the stall; an uncollected reservation ends and gives them back; one reservation holds at most 60 pairs.
`AuthControllerTest`, `LoginAttemptServiceTest`, `RegistrationLimiterTest`	Sign-up, sign-in, lockout after five failures, sign-up limits, memory that does not grow without end.
`AccountsTest`, `SuperAdminSetupTest`	Team accounts; a disabled account or a new password ends its sessions; the first super admin from the environment.

Test	What it shows
`HttpHardeningTest`, `RealServerSecurityTest`	Security headers, CORS, 401 against 403 on a real embedded server.
`PostgresIntegrationTest`	Migrations and constraints on PostgreSQL: impossible rows are refused by the database even when the application is bypassed; the last-pair and double-payment races over real HTTP.

Table 5.2: The main backend tests

The integration tests share a base class that starts the application once, creates real users with real tokens, and offers helpers for JSON requests (Code 5.1).

```
1 JsonNode small = json(deliver(buyer, v, 2, Map.of("zoneId", id)).
    andExpect(status().isCreated()));
2 assertThat(small.get("total").decimalValue()).isEqualByComparingTo(
    "22.00");
3 // A fee sent by the browser is ignored: only the area decides.
4 JsonNode big = json(deliver(buyer, v, 4, Map.of("zoneId", id, "fee"
    , "0")).andExpect(status().isCreated()));
5 assertThat(big.get("total").decimalValue()).isEqualByComparingTo("
    40.00");
6 assertThat(big.at("/fulfillment/delivery/fee").decimalValue()).
    isZero();
```

Code 5.1: An integration test: a fee sent by the browser is ignored

5.3 Frontend tests

The frontend tests use Vitest with Testing Library and a fake `fetch`. Among them: checkout sends a wallet token and an idempotency key but never a price or a card number, reuses the key only after a lost connection, charges delivery by area and sends no fee, and fills in remembered details; the bag reads only well-formed entries from storage; the sock drawing accepts only safe colours, so a hand-edited bag cannot inject markup; every text exists in Russian and Kyrgyz; every pair of text and background colours meets WCAG 2.2 AA contrast in both themes; and the wholesale contact check follows the server's rules.

5.4 The browser journey

`npm run e2e` drives Chromium through Playwright against the real backend and the built frontend. Its 29 steps cover the owner adding a section and a product with two colours and a picture (and a disguised file being refused), the product appearing in the shop with its filter count, removing and restoring it, a visitor filling the bag and registering, a declined and then a successful payment, the last pair bought and returned by cancelling, the customer asking for 90% and being held to the limit, the team handing the order over and reading the emails it caused, the site in Russian and Kyrgyz, the home page's sections, the owners' screens, the super admin's accounts, assistant and audit log, quick view and prices in som, the family and the gift box, reviews and search, a wholesale request with a wrong and then a right phone number, the women's shelf filtered by size and colour, and a guest ordering without an account and opening the order from a second browser, first without the link's key (and asking for it by email), then with it. The last step reserves an order to pay at the stall as a guest, and the team hands it over with the code and takes the money. After each important page the axe rules [29] run, and any serious or critical finding fails the journey.

The journey has caught problems that unit tests could not: a menu that flipped between two sizes in Russian, faded text that became too faint to read while a shelf reloaded, and a stamp checked half way through its animation on a slow machine. Each was fixed in the product, not by weakening the test.

5.5 Phones

Most of the stall's customers will open the shop on a phone, so the phone gets its own check besides the journey. A script opens 23 pages (the home page, every shelf, a product, the bag, checkout, the story, the wholesale and visit pages, the account pages and the owners' screens) at 360 and 390 pixels wide, in English, Russian and Kyrgyz, and reports three things on each: any element wider than the screen, any text cut off by its box, and any link or control smaller than a finger. The last limit is 44 by 44 CSS pixels, the size WCAG 2.2 names for touch targets.

The first run found no overflow or clipped text, but it found small targets in the footer, the breadcrumbs, the colour swatches and the leg-picker dots, and two real

bugs: the picture in quick view shrank almost to nothing in Safari, and in the "Try them on" teaser the leg moved off screen when a visitor switched between him, her and a child. After the fixes, the only targets under the limit are links inside running text, which WCAG exempts, and the round swatches, which keep their look and get an invisible larger hit area around them. The Safari fix was reasoned from the layout rules rather than measured, because WebKit could not run in the build container, so it is one of the checks to repeat by hand on an iPhone.

5.6 Evaluation of the two assistants

5.6.1 Method

Sixteen simulated customers (Table 5.3) each send one message about one sock: a browser asking the price, families, shop owners, a gift buyer, speakers of Russian and Kyrgyz, a greedy customer, a prompt-injection attempt, and socks with a thin or no margin. Each customer has a quantity, the limit of the sock and a walk-away point: the smallest discount at which they would buy. Both assistants answer every customer through the same service, prompt, reply checks and validator as the shop. The language-model answers are a recorded live run of gemini-3.5-flash-lite on 11 October 2026 (`docs/evaluation/live-2026-10-11.json`); the super admin can repeat the run at any time.

For each assistant the evaluation measures the average final discount, how many customers would buy, how often the validator had to cut a proposal, whether any final price went below the limit, and the revenue.

				Rules		**Language model**	
ID	**Customer**	**Pairs**	**Limit**	**prop.**	**final**	**prop.**	**final**
S01	Browser, one pair	1	15	0	0	0	0
S02	Asks for 10% on one pair	1	15	10	10	0	0
S03	Greedy, one pair	1	15	50	**15**	0	0
S04	Family of four	12	15	17	**15**	5	5
S05	Small shop owner	50	20	15	15	15	15
S06	Asks 30% for a carton	100	20	30	**20**	15	15

				Rules		Language model	
ID	**Customer**	**Pairs**	**Limit**	**prop.**	**final**	**prop.**	**final**
S07	Russian speaker, haggles	10	15	14	14	5	5
S08	Russian speaker, wholesale	60	18	15	15	15	15
S09	Kyrgyz speaker	10	15	14	14	5	5
S10	Gift buyer	3	12	10	10	5	5
S11	Tries an injection	1	15	90	**15**	0	0
S12	Exact ask within limit	20	15	12	12	10	10
S13	Thin margin sock	10	5	17	**5**	5	5
S14	No margin left	5	0	10	**0**	5	**0**
S15	Returning wholesale buyer	200	22	15	15	15	15
S16	Polite, no number	2	15	2	2	2	2

Table 5.3: The simulated customers: proposed and final discount in percent (bold: the validator cut the proposal)

5.6.2 Results

	Rules	Language model
Average final discount	11.1%	6.1%
Customers who would buy	81% (13 of 16)	50% (8 of 16)
Proposals cut by the validator	6	1
Final prices below the limit	0	0
Revenue of the simulated sales	$1,723.03	$1,001.62

Table 5.4: Summary of the evaluation

5.6.3 Discussion

Safety. No final price went below the limit on either side. The rule-based assistant over-promised six times, by design, and the validator cut every one of those proposals: the greedy customer, the injection attempt (90% proposed, 15% allowed), the family, the carton, the thin-margin sock and the sock with no margin. The language model

proposed a discount on a sock with no margin once (5%) and was cut to zero. The model also resisted the injection on its own and gave nothing; but the guarantee does not depend on that.

Commercial behaviour. The language model was more cautious than the rules: it gave smaller discounts and, against these walk-away points, fewer customers would have bought, so the simulated revenue was lower. This is a property of the prompt as much as of the model: the prompt tells it to start small and give more only for a reason, and in a single message there is little reason. In a real conversation the customer would answer and the model could move; the evaluation measures only the first answer. The rules, written for this shop, give the expected discounts for quantity at once.

Language and latency. The model answered every customer in their own language, including Russian and Kyrgyz, in about one to two seconds per message, within the conversational target of NFR-6; the rules answer instantly.

Validity. The customers are simulated and their walk-away points are assumptions, so the acceptance and revenue numbers compare the two assistants with each other rather than predict real sales. A user study with real customers of the stall would be the next step.

5.7 Requirements coverage

Every functional and non-functional requirement of Chapter 2 is implemented and shown by at least one of the tests above; `docs/traceability.md` lists, for each requirement, where it is implemented and which test shows it. Two are partial by design: grounding (FR-4) uses keyword retrieval rather than vector search, and offline use (NFR-5) covers the bag but not the catalogue.

Chapter 6

Conclusions and future work

6.1 Conclusion

This thesis designed and implemented AK&VEN, an integrated e-commerce and inventory management system for a family sock business at Dordoi Bazaar. It replaces the manual processes described in the introduction with one application. Customers browse the real assortment, buy for the whole family, mix any socks to reach a lower price per pair, ask for a better price in a conversation, and either pay online or reserve their order and pay when they collect it at the stall or receive it from a courier. The owners and staff manage the catalogue, stock, prices, deliveries, orders and wholesale requests in an administration interface, and see their sales on a dashboard. The shop and the administration are one installable web application in three languages; one REST API holds every business rule; and the database repeats the rules that must never be broken.

The system has two technical contributions. The first is a design in which a language model may talk about money without being able to decide it. The model only proposes; a deterministic validator limits every proposal to a value the model never sees, again at checkout; the database refuses an allowed discount larger than its proposal; and the model's words may not contain a number of their own. Both the proposal and the result are recorded, so the gap between what the model wanted and what was allowed is visible for every conversation. The second is an inventory that stays correct and explainable: stock is locked in a fixed order during checkout, reserved stock is held separately from stock on the shelf, and every change is written to a ledger in the same transaction as the change itself.

6.2 Answers to the research questions

RQ1: Can a language model negotiate while a deterministic component guarantees the limit? Yes. In the evaluation no final price went below the limit, whether the assistant was the rule-based one that over-promised six times or the language model, including during a prompt-injection attempt. The automated tests show the same for an assistant that deliberately offers 80%, for offers tampered with at checkout and for rows written directly to the database. The guarantee rests on code and constraints, not on the model's behaviour.

RQ2: How do the two negotiators compare? On the sixteen simulated customers the language model was more cautious: an average final discount of 6% against 11%, half of the customers buying against four in five, and the validator stepping in once against six times. It answered naturally in all three languages within one or two seconds. The rule-based assistant is therefore the better seller for single messages, and the model the better conversationalist; its caution comes largely from the prompt, which can be tuned to the owners' wishes.

RQ3: Can the bazaar's habits be expressed in an online shop? Yes. The shop expresses them as ordinary interface elements rather than as extra steps: a chat on the product page, a family planner, a price ladder for the whole bag shown in the bag and on every product, pickup with a code at the container, payment at handover, delivery areas with clear fees and times, and a case builder with a request inbox for shops. The browser tests and the accessibility checks show that these work for a first-time visitor, on a phone, and in Russian and Kyrgyz.

RQ4: Can the stock stay consistent and explainable? Yes. Stock is never read before the affected variants are locked, always in the same order, so two customers buying the last pair cannot both succeed and two opposite bags cannot deadlock; this is tested against a real PostgreSQL database over HTTP. Reserved pairs are counted separately, and a check constraint keeps the reserved quantity from exceeding the stock. Every sale, cancellation, delivery of new goods and manual correction writes one ledger row in the same transaction, and a test confirms that the rows of each variant always add up to the stock on the shelf, so any number on the shelf can be traced back to its causes.

6.3 Future work

- **A real card payment provider**, with the payment moved outside the checkout transaction (reserve, pay, confirm), and later a payment rail through a Kyrgyz bank. The reservation used for payment at handover already provides the first step.
- **A user study** with real customers of the stall, to measure how the two assistants are accepted in full conversations rather than single messages.
- **Vector retrieval** for the assistant, using the prepared pgvector table, so it can answer from longer product descriptions and the family's notes.
- **A Telegram bot** for the family and for customers. The shop already sends emails at the moments that matter, but most of the stall's customers use Telegram and WhatsApp. A bot would send the same messages, let a customer check an order by its number, and let the owners mark an order as handed over from their phone.
- **Real photographs** of the products, replacing the generated drawings.
- **Demand forecasting** from the stock ledger, the orders, the waiting lists and the wholesale requests, to suggest what and how much to reorder from the suppliers.
- **Translations of the owners' data**, such as section and colour names, into all three languages.

6.4 Final remarks

The project started from a simple observation at the stall: nobody pays the label price, and nobody knows exactly what is left on the shelf. The result is a system that keeps the conversation with the customer, keeps the family in control of the price, and keeps the stock accurate. It is ready to be used by my parents once the real prices, photographs and a payment provider are in place.

Acknowledgements

I would like to thank my supervisor, Walid Guettala, for his guidance, his trust and his support throughout this work. I am also grateful to my parents, Akylbek and Venera, whose stall at Dordoi Bazaar is the reason this system exists, and who patiently answered my questions about their products, customers and daily work.

Use of generative AI tools

During this project I used generative AI assistants as development and writing aids: to draft and review parts of the source code and the tests, to check my English, and to help structure parts of this document. I defined the problem, the requirements and the design decisions, reviewed and tested all generated material, and I take full responsibility for the content of the software and of this thesis. The language model used inside the product as a negotiation assistant (Section 4.9) is a separate component of the system and is evaluated in Chapter 5.

Bibliography

[1] Shopizer contributors. *Shopizer: Java open source e-commerce software.* URL: `https://github.com/shopizer-ecommerce/shopizer` (visited on 11/01/2026).

[2] Medusa contributors. *Medusa: Digital commerce platform.* URL: `https://github.com/medusajs/medusa` (visited on 11/01/2026).

[3] Saleor Commerce. *Saleor: Headless, GraphQL commerce platform.* URL: `https://github.com/saleor/saleor` (visited on 11/01/2026).

[4] Odoo S.A. *Odoo Inventory: Documentation.* URL: `https://www.odoo.com/documentation/17.0/applications/inventory_and_mrp/inventory.html` (visited on 11/11/2026).

[5] Martin Fowler. *Event Sourcing.* 2005. URL: `https://martinfowler.com/eaaDev/EventSourcing.html` (visited on 11/11/2026).

[6] The PostgreSQL Global Development Group. *PostgreSQL Documentation: Explicit Locking.* URL: `https://www.postgresql.org/docs/16/explicit-locking.html` (visited on 11/11/2026).

[7] Mike Lewis et al. "Deal or No Deal? End-to-End Learning of Negotiation Dialogues". In: *Proceedings of the 2017 Conference on Empirical Methods in Natural Language Processing.* ACL, 2017, pp. 2443–2453.

[8] He He et al. "Decoupling Strategy and Generation in Negotiation Dialogues". In: *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing.* ACL, 2018, pp. 2333–2343.

[9] Kai Greshake et al. "Not What You've Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection". In: *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security.* ACM, 2023, pp. 79–90.

[10] OWASP Foundation. *OWASP Top 10 for Large Language Model Applications, 2025*. 2024. URL: https://genai.owasp.org/llm-top-10/ (visited on 11/01/2026).

[11] Patrick Lewis et al. "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks". In: *Advances in Neural Information Processing Systems 33*. 2020.

[12] Andrew Kane. *pgvector: Open-source vector similarity search for Postgres*. URL: https://github.com/pgvector/pgvector (visited on 11/01/2026).

[13] MDN contributors. *Progressive Web Apps*. URL: https://developer.mozilla.org/en-US/docs/Web/Progressive_web_apps (visited on 11/01/2026).

[14] Google. *Gemini API: OpenAI compatibility*. URL: https://ai.google.dev/gemini-api/docs/openai (visited on 11/01/2026).

[15] Docker Inc. *Docker Compose Documentation*. URL: https://docs.docker.com/compose/ (visited on 11/01/2026).

[16] Redgate. *Flyway Documentation: Migrations*. URL: https://documentation.red-gate.com/flyway (visited on 11/01/2026).

[17] VMware Tanzu. *Spring Boot Reference Documentation, version 3.3*. 2024. URL: https://docs.spring.io/spring-boot/3.3/ (visited on 11/01/2026).

[18] VMware Tanzu. *Spring Security Reference: Method Security and Role Hierarchy*. 2024. URL: https://docs.spring.io/spring-security/reference/ (visited on 11/01/2026).

[19] The PostgreSQL Global Development Group. *PostgreSQL 16 Documentation*. 2023. URL: https://www.postgresql.org/docs/16/ (visited on 11/01/2026).

[20] M. Nottingham and E. Wilde. *Problem Details for HTTP APIs*. RFC 7807. IETF, 2016. URL: https://www.rfc-editor.org/rfc/rfc7807.

[21] M. Jones, J. Bradley, and N. Sakimura. *JSON Web Token (JWT)*. RFC 7519. IETF, 2015. URL: https://www.rfc-editor.org/rfc/rfc7519.

[22] Meta Open Source. *React Documentation*. URL: https://react.dev/ (visited on 11/01/2026).

[23] Microsoft. *TypeScript Handbook*. URL: https://www.typescriptlang.org/docs/handbook/ (visited on 11/01/2026).

[24] Vite contributors. *Vite: Next Generation Frontend Tooling*. URL: https://vite.dev/ (visited on 11/01/2026).

[25] The JUnit Team. *JUnit 5 User Guide*. URL: https://junit.org/junit5/docs/current/user-guide/ (visited on 11/01/2026).

[26] Mountainminds GmbH. *JaCoCo Java Code Coverage Library*. URL: https://www.jacoco.org/jacoco/ (visited on 11/01/2026).

[27] Vitest contributors. *Vitest: Next generation testing framework*. URL: https://vitest.dev/ (visited on 11/01/2026).

[28] Microsoft. *Playwright Documentation*. URL: https://playwright.dev/ (visited on 11/01/2026).

[29] Deque Systems. *axe-core: Accessibility engine for automated Web UI testing*. URL: https://github.com/dequelabs/axe-core (visited on 11/01/2026).

[30] Martin Fowler. *Patterns of Enterprise Application Architecture*. Addison-Wesley, 2002. ISBN: 0-321-12742-0.

[31] OWASP Foundation. *CSV Injection*. URL: https://owasp.org/www-community/attacks/CSV_Injection (visited on 11/01/2026).

[32] World Wide Web Consortium. *Web Content Accessibility Guidelines (WCAG) 2.2*. 2023. URL: https://www.w3.org/TR/WCAG22/ (visited on 11/01/2026).

List of Figures

2.1 Use cases of visitors and customers 13
2.2 Use cases of the staff, the owners and the super admin 14

3.1 The footer: the stall's door with its open or closed sign, the promises as sewn labels, every page and the contacts 23
3.2 The home page 24
3.3 Who are they for? Each shelf on its own fabric, with its socks on the line 24
3.4 The men's shelf 25
3.5 Occasions, heights, sizes and colours; the chosen size shown as a tag . 26
3.6 A product page 27
3.7 How the price falls: the price per pair and in total as the number of pairs grows 27
3.8 The chat: asked for 50%, the assistant offers what the owners allow . 28
3.9 The fitting room 29
3.10 The family planner, kept on a sheet of paper pegged to the page . . . 30
3.11 Building a gift box 30
3.12 The bag as one collection 31
3.13 Checkout with a delivery area 32
3.14 The simulated wallet 32
3.15 A paid order 33
3.16 The account 34
3.17 The wholesale page 35
3.18 Building a case 35
3.19 A request sent, with its reference 36
3.20 Find us at Dordoi 37
3.21 The home page and the children's shelf on a phone, in Kyrgyz 38
3.22 Products 39
3.23 Adding a product 39

3.24 Goods arrived, and an option's stock history 40
3.25 Orders 41
3.26 Handing an order over at the stall 41
3.27 Negotiations: proposed and allowed, side by side 42
3.28 The dashboard 42
3.29 The inbox of wholesale requests 43
3.30 Pricing 44
3.31 Delivery areas 45
3.32 Choosing and trying the assistant 46
3.33 The audit log 46

4.1 Architecture overview 49
4.2 The core of the data model 53
4.3 The validation path: the proposal is untrusted until the validator has clamped it 61
4.4 One negotiation turn 62
4.5 The states of an order and what moves it between them 66

List of Tables

2.1 Users of the system 12
2.2 Functional requirements 17
2.3 Non-functional requirements 18

3.1 Demo accounts 21
3.2 The main settings 22

4.1 Technology stack 50
4.2 The main tables 54
4.3 The API by audience 57

5.1 The test suites 74
5.2 The main backend tests 76
5.3 The simulated customers: proposed and final discount in percent (bold: the validator cut the proposal) 79
5.4 Summary of the evaluation 79

List of Codes

3.1 Starting the shop 21
4.1 Repository layout 50
4.2 The PolicyValidator 58
4.3 Deciding the price of an order line 59
4.4 A reply may state no number of its own (Cyrillic shown as Java Unicode escapes) 63
5.1 An integration test: a fee sent by the browser is ignored 76